Canary SSM

Change plan, validation programme and paper-writing kit

Revision 3. Written after the first full-scale run by Antigravity (final benchmark, Experiment 7, Experiment 8) on the user's PC. This document tells the coding agent exactly what to change in the current implementation, which experiments to run next, how to run them, how to analyse them, and how to turn the results into a research paper.

Item	Status at the time of writing
Project	Online detection of weak, persistent latency regressions on an unknown subset of canary hosts under shared workload change; hierarchical state-space model versus strong sequential baselines at equal false-alert rate
Current verdict	NOT GO. Pipeline works at scale; calibration gate fails (shared-step regime); comparative claims are confounded by protocol flaws that this document fixes
Evidence base	Experiments 1-6 (second-hand handoffs), my 200-rollout and 500-rollout null audits, and Antigravity's final benchmark with Experiments 7 and 8 (summary package inspected; raw artifacts not yet seen)
Scope of claims	Simulation results under declared assumptions, plus (if Part 4 succeeds) semi-synthetic results on real traces. No causal attribution, no production claims, no novelty claims without a literature check
Audience	(1) The user; (2) Antigravity, which executes the work packages and experiments; (3) whoever reviews the paper

Note. How to read the numbers in this document. Numbers quoted from the pilot handoffs are 'as reported'. Numbers quoted from the Antigravity package were read from the saved CSV and JSON files. Numbers I computed myself in a sandbox are labelled as such. Anything described as an estimate (runtimes, effort) is an estimate, not a measurement.

Contents

Part 0. How to use this document 6

0.1 Rules of engagement for Antigravity . . . 6

0.2 Definition of done for any work package . . . 6

0.3 Frozen definitions (do not change silently) . . . 6

Part 1. Where the project stands and why 8

1.1 What exists . . . 8

1.2 What the full-scale run showed . . . 8

Positive findings (exploratory, simulation only) . . . 8

Negative or unresolved findings . . . 8

1.3 Status against the plan . . . 9

1.4 Issue register . . . 9

1.5 Root-cause analysis of the headline contradiction . . . 10

1.6 Decision tree for the research story . . . 11

1.7 Phases and dependencies . . . 11

Part 2. Change specifications for the current implementation 12

Group A. Protocol, configuration, manifest, seeds . . . 12

WP-A1. Separate the plain and step regimes (fixes I-01) . . . 12

WP-A2. Per-family validation sized for a valid gate (fixes I-02, replaces my earlier gate) . . . 13

WP-A3. Complete the manifest (fixes I-10) . . . 13

WP-A4. Central seed registry (fixes I-14) . . . 14

WP-A5. Use the pipeline's eligibility everywhere (fixes I-13) . . . 14

WP-A6. Move experiment scripts into the package (fixes I-05) . . . 14

Group B. Grids, selection objective, ablation fairness . . . 14

WP-B1. Declared grids as data, extended and budget-matched (fixes I-03, I-15 partly) . . . 14

WP-B2. Automatic edge-of-grid detection and grid expansion protocol (fixes I-03) . . . 15

WP-B3. Cross-validated selection with a one-standard-error rule (improves select_on_dev) . . . 15

WP-B4. Fair no-persistence ablation (fixes I-04) . . . 16

WP-B5. Localization cutoff range and selection (fixes I-20 partly) . . . 16

WP-B6. Frozen versus tuned guard (fixes I-05) . . . 16

Group C. Calibration methodology . . . 16

WP-C1. Policy schema v2 with regimes . . . 16

WP-C2. Sample size, multiple alphas and threshold uncertainty (fixes I-18) . . . 16

WP-C3. Exact validity test replacing the ad-hoc gate (fixes I-07 diagnosis) . . . 17

WP-C4. Horizon consistency . . . 17

WP-C5. Recalibration command and calibration-transfer measurement (fixes I-08) . . . 17

WP-C6. Small-sample recalibration (how little healthy history is enough) . . . 18

Group D. Model and estimator improvements . . . 18

WP-D1. Empirical-Bayes slope and loading shrinkage using the controls (fixes I-22, I-23) . . . 18

WP-D2. Noise parameters from the controls: aim for a pivotal statistic (fixes I-08, I-11) . . . 19
WP-D3. Parameter-uncertainty-aware filter (regression-state SSM, RS-SSM) (fixes I-22) . . . 19
WP-D4. Support-based regime statistic replacing the simulator-tuned selector (fixes I-06) . . . 20
WP-D5. Robust observation handling for heavy tails (fixes part of I-11) . . . 21
WP-D6. Factor-shock sensitivity and per-host noise scale (fixes part of I-11) . . . 21
WP-D7. Release score alternatives . . . 21
WP-D8. Hypothesis-enumeration Bayesian detector (targets the unknown affected subset; fixes I-12) . . . 22
WP-D9. Structure-averaged detector (cheaper cousin of D8) . . . 22
Group E. Baselines and reference detectors . . . 23
WP-E1. Additional baselines . . . 23
WP-E2. Clairvoyant reference detectors (upper bounds, never competitors) . . . 24
WP-E3. Tuning-budget parity and fairness protocol . . . 24
Group F. Metrics, statistics and reporting . . . 24
WP-F1. Pre-registration and the contrast registry . . . 24
WP-F2. Paired inference with multiplicity control . . . 24
WP-F3. Time-to-detection analysis (survival view) . . . 25
WP-F4. Power versus false-alert curves . . . 25
WP-F5. Host-level inference: localization, FDR and probability calibration . . . 25
WP-F6. Decision-theoretic evaluation (operational loss) . . . 25
WP-F7. Alert flicker and advisory policy . . . 25
WP-F8. Stratified and sensitivity reporting . . . 26
WP-F9. Report generator hardening . . . 26
Group G. Simulator realism . . . 26
Group I. Engineering, reproducibility and speed . . . 27
WP-I1. Version control, locking, task runner . . . 27
WP-I2. Parallel runner, caching and resume (fixes I-17) . . . 27
WP-I3. Test-suite expansion . . . 27
WP-I4. Numerical diagnostics . . . 28
WP-I5. Documentation . . . 28
WP-I6. On-the-fly data generation and storage budget . . . 28
Part 3. Validation and experiment programme **29**
Tier 0. Repair the evidence base . . . 29
X-0.1 Reproduction and determinism . . . 29
X-0.2 Explain the two anomalies and the hier = label-error invariance . . . 29
X-0.3 Grid expansion on development data . . . 29
X-0.4 Plain-regime calibration validity (the gate that opens Tier 1) . . . 30
Tier 1. The fair benchmark and ablations . . . 30
E1 Main benchmark in the plain regime . . . 30
E2 Ablation suite . . . 31
E3 Hyperparameter sensitivity and tuning-budget parity . . . 31
E4 Power surfaces and minimum detectable effect . . . 31
E5 Power versus false-alert curves . . . 32

E6 Host localization, FDR and probability calibration . . . 32
E7 Decision-theoretic evaluation . . . 32
E8 Advisory policy and flicker . . . 32
Tier 2. Robustness studies (the failure map) . . . 32
R1 Shared workload steps, ramps and slope heterogeneity (the main failure) . . . 32
R2 Calibration transfer: noise persistence, heavy tails, loading shifts, nonlinearity, white noise . . . 33
R3 Control contamination and interference . . . 34
R4 Missingness and abstention . . . 34
R5 Shared incidents and negative controls . . . 34
R6 Effect shape misspecification: ramps, flapping, load-dependent harm, improvements . . . 34
R7 Scalability and numerical behaviour . . . 35
R8 Class structure and metadata quality . . . 35
R9 Multi-metric extension . . . 35

Part 4. Real-trace validation **36**

4.1 Candidate sources to inspect . . . 36
4.2 From a trace to the Rollout schema . . . 36
4.3 Placebo calibration check on real nulls (RT-1) . . . 36
4.4 Semi-synthetic injection (RT-2) . . . 37
4.5 Simulator fidelity table (RT-3) . . . 37
4.6 Rules specific to real data . . . 37

Part 5. Paper kit **38**

5.1 What can be a paper, and how to decide . . . 38
5.2 Title and abstract templates (fill only from generated numbers) . . . 38
5.3 Section outline with source of each claim . . . 38
5.4 Figures (each generated from CSV by report.py; vector PDF; CSV twin saved) . . . 39
5.5 Tables . . . 39
5.6 Theory notes to write (each verified numerically before it is claimed) . . . 39
5.7 Related work to read and verify (do not cite from memory) . . . 40
5.8 Claims discipline: allowed and disallowed statements . . . 40
5.9 Threats to validity . . . 41
5.10 Reproducibility package . . . 41
5.11 Writing workflow . . . 42
5.12 Venue options (hedged) . . . 42

Part 6. Execution run book **43**

6.1 Order of work . . . 43
6.2 Compute plan for the PC (12 threads, 16 GB RAM) . . . 43
6.3 Risk register . . . 44
6.4 If-then decisions . . . 44

Part 7. Prompt library for Antigravity **45**

P-00 Standing instruction (paste first in every new session) . . . 45

Housekeeping and protocol . . . 45

Methods and baselines . . . 45

Statistics, simulator and studies . . . 46

Appendices **47**

Appendix A. Mathematical reference . . . 47

A.1 Kalman filter (Joseph form, current implementation) . . . 47

A.2 Threshold and exact validity . . . 47

A.3 Binomial intervals . . . 47

A.4 Empirical-Bayes slope shrinkage (WP-D1) . . . 47

A.5 Regression-state SSM (WP-D3) . . . 47

A.6 Support statistics (WP-D4) . . . 47

A.7 Hypothesis enumeration (WP-D8) . . . 47

A.8 Sequential statistics for baselines (verify against sources) . . . 47

A.9 Inference for comparisons . . . 47

A.10 Survival and loss . . . 48

Appendix B. Configuration examples . . . 48

Appendix C. Command-line map . . . 48

Appendix D. Result file dictionary (columns that must exist) . . . 49

Appendix E. Checklists . . . 49

E.1 Before any run . . . 49

E.2 Before reading any result . . . 49

E.3 Before writing a claim . . . 49

E.4 Before submission . . . 49

Appendix F. Glossary . . . 50

Appendix G. Pre-registration template (docs/PREREG.md) . . . 50

Appendix H. Repository map after the changes . . . 50

Appendix I. Troubleshooting . . . 50

Part 0. How to use this document

This document has seven parts. Part 1 explains where the project stands and why. Part 2 specifies every change to the current code, one work package at a time, with file names, function names, algorithms, tests and commands. Part 3 specifies the experiment programme, one experiment at a time, with hypotheses, design, protocol, analysis and decision rules. Part 4 covers real-trace validation. Part 5 is the paper kit: storyline decision tree, section outline, figure and table list, theory notes, related work to verify, threats to validity. Part 6 is the execution run book with time estimates and gates. Part 7 is the Antigravity prompt library and the appendices (mathematics, configuration examples, CSV dictionary, checklists).

0.1 Rules of engagement for Antigravity

These rules apply to every work package and experiment. Paste them as the standing instruction at the start of each session.

1. **Do one work package or experiment at a time.** Finish it, run its acceptance tests, report, and stop. Do not start the next one without an instruction.
2. **Never modify an existing test to make it pass.** If a test seems wrong, stop and explain. New tests may be added freely. Every bug fix gets a regression test that fails before the fix.
3. **Never invent numbers.** Every number in any report must be produced by a script from saved CSV files. Prose summaries are generated from tables by code, never typed.
4. **PASS and FAIL are computed, not typed.** The calibration gate, acceptance criteria and pre-registered contrasts are evaluated by code and written to JSON.
5. **Data roles never mix.** Development data fits and tunes. Calibration data sets thresholds. Validation data audits. Test data is touched once per frozen model version. Stress and robustness data is never used to tune.
6. **Detector code never reads ground truth.** Modules factor, preprocess, baselines, kalman, hierarchy, decision, and every new detector module may not import simulate.Truth, metrics, or any *_truth file. A test enforces this and must be extended to every new module.
7. **Causality.** The score at time t uses data up to and including time t only. Any statistic that looks ahead (including a regime selector) may not influence alarms before the lookahead window has elapsed.
8. **Tuned and frozen models are different artifacts.** A model produced with --frozen exists only for calibration audits. Any script that reports detection performance must refuse a frozen model.
9. **Record everything.** Every run writes a manifest: git commit, configuration hash, seeds used, beta mode, grids, selected hyperparameters, package versions, machine, timings, failures.
10. **Report failures.** Fit failures, boundary hits, non-convergence, ineligible windows, and abstentions are reported, never dropped.
11. **Seeds come from the registry** (work package A4). No script may invent a seed range.
12. **Ask before changing a definition.** The harm boundary, event definition, outcome classes, calibration rule and primary metric (Section 0.3) change only by explicit decision, with the change logged in docs/DESIGN_DECISIONS.md.

0.2 Definition of done for any work package

- Code merged on a branch named wp-ID, with tests added and the whole suite green (python -m pytest -q).
- A short report file docs/wp/WP-ID.md containing: what changed, files touched, commands run, test output, any deviation from this document and why.
- If the work package changes statistical behaviour, a before-and-after table generated by a script (not typed).
- The manifest of any run it produced.

0.3 Frozen definitions (do not change silently)

Item	Definition
Metric and units	y = log latency; one-minute windows; 180 pre-rollout windows and 120 monitored windows by default; latency = exp(y)
Harm boundary	$\kappa = \log(1.02) \approx 0.01980$. An effect equal to κ is not harmful. 3% latency is $\log(1.03) = 0.02956$, not 0.03
Qualifying event	$J(i,t) = 1$ iff effect $a(i,t) > \kappa$ strictly. An event is a run of at least $D = 15$ consecutive minutes on at least one canary. Onset = start of the first qualifying run. Truth only; detectors never see it
Alarm	First eligible minute at or after t0 whose score strictly exceeds the method threshold. Alarms are latched per rollout
Outcome classes	success (alarm inside a qualifying interval at or after onset), premature (before onset), miss (no alarm), late after recovery (alarm after all qualifying intervals ended)
Primary metric	Horizon-capped time to success: delay for successes, remaining horizon from onset for every other outcome. Reported with miss, premature and late rates. Conditional delay alone is never reported
Threshold rule	B null maxima over the full monitoring horizon; $k = \text{ceil}((B+1)(1-\alpha))$; alert when a future maximum strictly exceeds the k-th smallest; infinite if k exceeds B. B=500, α=0.05 gives k=476; B=1000 gives k=951
Calibration gate (G2)	A method passes a cell if the exact beta-binomial validity test (WP-C3) gives a two-sided p-value of at least 0.01 AND the achieved false-alert rate is at most 0.075. The final gate uses B at least 2000 calibration nulls and at least 2000 validation nulls per family. Clopper-Pearson intervals are reported for readers. The gate is never PASS without at least one evaluable cell. (This replaces my earlier rule 'upper bound at most 0.08 with n at least 400', which a perfectly valid method fails 29% of the time at B = n = 500.)
Release score (SSMs)	log of the maximum over canary hosts of $\Pr(a_i > \kappa \mid \text{data up to } t)$. A ranking score, not the probability that any host is harmful. Its threshold is empirical
Comparison rule	Delay, power and localization are compared only between methods that pass the calibration gate in the regime being compared

Two observations from the audit motivate the last row of this table. First, a method with a looser achieved false-alert rate looks faster, so equal-rate comparison is the only fair one. Second, the pilot reports repeatedly labelled a method PASS while its own table showed an interval that excluded 5%, so verdicts must come from code.

Part 1. Where the project stands and why

1.1 What exists

The repository canary_ssm_project implements the statistical track of the plan: a paired null/harm simulator with isolated ground truth (simulate.py, schema.py); a control-derived rank-one factor adjustment with per-host workload regression (factor.py, preprocess.py); a Joseph-form Kalman filter verified against statsmodels (kalman.py); independent, pooled and hierarchical effect state-space detectors with AR noise states (hierarchy.py); adjusted CUSUM, EWMA, top-K CUSUM and matched-contrast baselines (baselines.py); an order-statistic threshold calibrator with latched alarms (decision.py); event labelling, outcome classification, localization, Brier score and exact intervals (metrics.py); orchestration for simulate, fit, calibrate, validate-null, audit-null, evaluate and benchmark (experiments.py, cli.py); and report generation with a numeric calibration gate (report.py). Antigravity ran this code unchanged (one Windows encoding fix) and added two scripts, experiment_7_causal_regime.py and experiment_8_robustness.py.

1.2 What the full-scale run showed

Positive findings (exploratory, simulation only)

- Aggregate calibration on the 500-null benchmark mixture is correct: all 16 methods alert on 2.0% to 4.8% of nulls, all upper bounds at most 8%.
- Mean horizon-capped time on 474 qualifying events: hierarchical SSM 27.1 minutes, independent SSM 27.3, pooled SSM 56.1, top-K CUSUM 53.0, matched contrast 69.4, EWMA 70.9, adjusted CUSUM 86.0. These are confounded (Section 1.4).
- Paired differences (hier minus comparator, negative favours hier): versus pooled −29.0 [−33.8, −24.5]; versus top-K CUSUM −25.8 [−28.9, −22.9]; versus no factor −44.0; versus white-only noise −6.3 [−8.8, −3.7]; versus independent hosts −0.18 [−2.9, +2.7] (a tie); versus 20%-label-error variant +0.10 [−1.8, +2.0] (a tie).
- Structure matters conditionally. Class-wide harm: success 1.00 (hier) versus 0.91 (independent). Sparse harm: 0.52 versus 0.70 (weak sparse) and 0.65 versus 0.84 (single canary).
- Host-level probabilities are best calibrated for the hierarchical model (Brier 0.031 versus 0.052 independent and 0.053 pooled).
- Runtime is not a constraint: 99th percentile update 0.40 ms at 10 canaries, 54 ms at 100 (Intel 11th-generation class CPU).

Negative or unresolved findings

- Calibration gate: FAIL (117 of 128 cells ok). All 11 failing cells are the step family (n=79 nulls). Step nulls trigger adjusted CUSUM 27.9%, top-K CUSUM 24.1%, independent SSM 17.7%, EWMA 17.7%, hierarchical SSM 11.4%, matched contrast 3.8%, pooled SSM 2.5%.
- Frozen-parameter audit at n=500 per cell (calibrated on plain nulls): step-only nulls trigger adjusted CUSUM 66.4%, hierarchical SSM 14.6%, matched contrast 11.6%. Nonlinear workload: CUSUM 78.2%, hierarchical 26.8%. Persistent noise φe=0.8: hierarchical 24.4%, CUSUM 34.6%. Loading shift ×1.3: hierarchical 10.2%, independent 13.2%.
- Experiment 7 (frozen model, pooled slope): single-threshold false alerts on plain nulls were only 1.0% (CUSUM) and 0.8% (top-K), proving that a mixture threshold contaminated by step nulls throttles the baselines. With clean plain calibration top-K CUSUM detected 79.5% versus hier 41% (but with K equal to the true affected count and untuned hier). A dual-regime rule still left step-regime false alerts at 9.0% (CUSUM), 7.6% (top-K), 6.8% (hier), and CUSUM step-regime power fell to about 1%.
- Experiment 8: per-host pre-period slopes give the smallest per-rollout residual shift SD (0.0152 versus 0.0265 pooled and 0.0272 matched contrast). Huber clipping lowered Student-t(3) false alerts from 12.8% to 9.5% (power 32.7% to 28.0%). A 'conservative' φe=0.8 filter lowered persistent-noise false alerts from 25.0% to 17.8%; both remain far above 5%.

1.3 Status against the plan

Plan stage	State	Comment
0 Protocol, schema, manifest	Done, incomplete manifest	beta_mode, grids, commit hash, lockfile missing (I-10)
1 Simulator	Done	Gaussian, single metric, matched to the model (I-16)
2 Adjustment and baselines	Done, baselines under-tuned	Grids hit edges; no strong sparse comparator (I-03, I-15)
3 Independent / pooled SSM	Done	
4 Hierarchical SSM	Done	Ties independent hosts overall (I-12)
5 Calibration, validation, evaluation, ablations	Run once; gate FAILED	Benchmark executed before the gate passed; treat as exploratory
6 Robustness and stress	Partly (Exp 7, 8, audit)	Calibration not robust to any misspecification tested (I-11)
7 Packaging and paper	Not started	This document

In one sentence: the build phase is complete, we are in the repair-and-validate phase, and no comparative claim is yet defensible. The first full benchmark should be treated as a dress rehearsal that exposed protocol flaws, which is a normal and useful outcome.

1.4 Issue register

Each issue has an identifier used throughout this document. Severity: A blocks any claim; B weakens claims; C is hygiene. 'WP' means the work package in Part 2 that fixes it; 'X' means the experiment in Part 3 that tests it.

ID	Sev	Issue	Evidence	Fix
I-01	A	Step nulls (15% weight) sit inside the calibration and validation mixture, inflating thresholds	configs/default.yaml family S1_step; CUSUM threshold 53.0; Exp 7 single-threshold plain false alerts 1.0% (CUSUM), 0.8% (top-K); S1_sparse CUSUM false alerts 0 of 108	WP-A1, WP-C1
I-02	A	Calibration gate cannot be evaluated: per-family cells hold 26 to 108 nulls	gate_status.json: evaluable_cells = 0	WP-A2
I-03	A	Selected hyperparameters sit at grid edges: ρ=0.99 and sd=0.001 for all three SSMs; no-persist sd=0.01; CUSUM k=0.5; EWMA α=0.05; top-K K=5; localization cutoff 5	experiment_manifest.json	WP-B1, WP-B2
I-04	A	No-persistence ablation is structurally handicapped: effect prior SD at most 0.01 cannot reach the 0.0296 effect	model_config: hier_no_persist sd_g=0.01 (max of grid)	WP-B4
I-05	A	Experiments 7 and 8 used the frozen audit model (hier ρ=0.95; top-K K=3, equal to true affected count) and pooled slope; main run used tuned model and per-host pre-period slope	scripts read data_audit/model_pooled_dev	WP-A6, WP-B6
I-06	B	Experiment 7 regime selector is hard-coded (threshold 0.17 versus a simulated step of exactly 0.3365 at exactly t0), uses x up to t0+4 for alarms from t0, and its accuracy was not saved	experiment_7_causal_regime.py	WP-D4, X-R1
I-07	B	Per-regime calibration left step-regime false alerts at 6.8% to 9.0% though exchangeability predicts about 5%	table_regime_null_rates.csv	WP-C2, X-0.2
I-08	B	Experiment 8 'conservative φ' and Huber tests apply a threshold calibrated on nominal data to misspecified data: a calibration-transfer test, not a filter test	experiment_8_robustness.py, cal_rolls from SimConfig(sd_host=0.02)	X-R2, WP-C5, WP-C6
I-09	B	hier_arwhite results are numerically identical to hier_ssm (same miss rate 0.162, same capped time 27.129)	table_detection.csv	X-0.2

ID	Sev	Issue	Evidence	Fix
I-10	B	Manifest omits beta_mode, grids, candidate table hash, commit (null), lockfile (null)	experiment_manifest.json	WP-A3, WP-I1
I-11	A	Calibration is not robust to shared step, nonlinear workload, persistent noise, heavy tails, loading shift	audit tables (n=500)	WP-C, WP-D, X-R1, X-R2
I-12	B	Hierarchy ties independent hosts overall and loses in sparse harm	paired differences; success by family	WP-D9
I-13	C	Experiment 8 uses decision_obs.all(axis=1) as eligibility rather than the pipeline's eligibility	script lines in Part 2 and 3	WP-A5
I-14	C	Ad-hoc seed ranges in experiment scripts; no disjointness guarantee	2_100_000, 2_600_000, 7_500_000, 4_100_000, 6_300_000 ...	WP-A4
I-15	A	Baseline suite is thin: no windowed GLR, no BOCPD, no always-valid sequential test (mSPRT), no Xie-Siegmund mixture GLR, no rank-based canary judge, no clairvoyant reference	baselines.py	WP-E1, WP-E2
I-16	A	Single Gaussian metric, AR(1) noise, constant-magnitude step effect: the simulator matches the filter's own assumptions	simulate.py	WP-G, Part 4
I-17	C	Single-process execution; adjusted residuals recomputed for every candidate; fit takes tens of minutes	experiments.py select_on_dev	WP-I2
I-18	B	B=500 gives a noisy 95th-percentile threshold; no α sweep; no threshold uncertainty reported	calibrate()	WP-C2, WP-F4
I-19	B	No pre-registered contrast list; no multiplicity control; paired contrasts not stratified by scenario and noise	table_paired_differences.csv	WP-F1, WP-F2
I-20	B	Localization cutoff at edge; no host-level FDR reporting; reliability curves lack intervals	model_config loc_cutoff=5	WP-F5
I-21	B	Noise-sweep and stress outputs were not in the summary package (unknown whether run)	package contents	Request files; X-E1
I-22	A	Plug-in adjustment ignores parameter uncertainty: per-host intercept and slope errors become persistent offsets that look like harm (≈±0.2σ intercept error; slope error × step)	my null audits; Exp 8 beta error RMS 0.053 per-host, 0.080 pooled	WP-D1, WP-D3
I-23	B	Pre-period of 180 minutes limits slope identification (workload variance 0.0095)	Exp 8; arithmetic: SE(β) ≈ σ/√(T·Var x)	X-R1 (pre-period sweep), WP-G6
I-24	B	Regime-conditional policy and power cost not characterised across step sizes, ramps and onset offsets	Exp 7 covers one step size at one onset	X-R1

1.5 Root-cause analysis of the headline contradiction

The main benchmark says SSMs are far faster than CUSUM. Experiment 7 says top-K CUSUM beats the hierarchical SSM in the plain regime. Both are produced by the same codebase, so the contradiction is a protocol artefact. Five mechanisms contribute, and each is addressed separately.

1. **Threshold contamination (I-01).** The calibration mixture contains step nulls on which CUSUM false-alerts at about 28%. The 95th percentile of the mixture's maxima therefore moves into the step-null range. CUSUM then has a far higher threshold than it needs on plain nulls and loses power. The SSMs are less affected (hier 11% on step nulls), so the contamination favours them.
2. **Tuning asymmetry (I-03).** Baseline hyperparameters were selected at the edge of narrow grids, which usually means the optimum lies outside. SSMs were also at edges but their edge (ρ=0.99, sd=0.001) is closer to the generative optimum for a constant persistent shift.
3. **Oracle leakage in Experiment 7 (I-05).** The frozen top-K detector uses K=3 and the scenario has exactly 3 affected canaries. That is information the main fit does not have. The frozen hier uses ρ=0.95, which is not what tuning selects.

4. **Generative alignment (I-16).** A persistent, constant-size, Gaussian-noise shift is exactly what a high-persistence linear-Gaussian filter is built for. A well-tuned CUSUM is close to optimal for a known shift size. The question is how close, which only a fair protocol can answer.
5. **Parameter uncertainty (I-22).** Plug-in baselines and SSMs both inherit per-host offsets from the pre-period fit. This hurts CUSUM more because it accumulates.

The fair protocol is: calibrate every method on plain nulls only; tune every method over a comparable number of candidates on a development set that never sees the test scenarios' truth; extend grids until the selection is interior; select K on development data; include clairvoyant reference detectors to bound what is achievable; compare at equal validated false-alert rates; report the step regime separately with its own calibration.

1.6 Decision tree for the research story

The fair re-benchmark (experiment E1) decides which paper is writable. Decide this before writing, and pre-register the decision rule.

Outcome of E1 (plain regime, equal validated false-alert rate)	Paper that follows
SSM family beats the best tuned baselines by more than the pre-registered margin; hierarchy beats independent in class-wide harm; a structure-adaptive extension (WP-D9) fixes sparse harm	Method paper: structure-adaptive state-space canary analysis, with a calibration-robustness section
SSM family beats baselines but calibration is fragile and no robust fix is found	Empirical study: a benchmark and audit of canary analysis under shared workload change, with the recommended calibration protocol and failure map
Tuned CUSUM/GLR is competitive (differences inside the margin)	Negative-result or benchmark paper: model complexity is not justified in simulation; contribute the simulator, the protocol, the audit, and the failure modes
Results flip between plain and real-trace placebo evaluation	Realism paper: the simulator's assumptions are the story; report which assumptions matter (ablate them one at a time)

All four are publishable if the evidence is clean. What is not publishable is a claim that outruns the protocol, which is why Parts 2 and 3 spend most of their length on protocol.

1.7 Phases and dependencies

Phase	Content	Depends on	Gate to leave the phase
P0 Housekeeping	WP-A3, A4, I1 (git, lockfile, seeds, manifest, CI script)	none	Clean-environment test run reproduces; manifest complete
P1 Protocol repair	WP-A1, A2, A5, A6, B1-B6, C1-C4; experiments X-0.1 to X-0.4	P0	Gate G2 (Section 0.3) PASS for every baseline and SSM in the plain regime on 2000 calibration nulls and 2000 validation nulls per family; anomalies I-07 and I-09 explained
P2 Fair benchmark	WP-E1, E2, F1-F6; experiments E1 to E8	P1	Pre-registered contrasts computed; decision-tree branch chosen
P3 Robustness and new methods	WP-C5, C6, D1-D9; experiments R1 to R9	P1 (some in parallel with P2)	For each stress family: calibration transfer characterised with the G2 validity test; at least one mitigation evaluated at n of at least 1000 nulls per cell
P4 Realism	WP-G, semi-synthetic real traces, placebo calibration (Part 4)	P1	Placebo false-alert rate and injected-effect power reported on at least one real trace set
P5 Paper	Part 5: figures, tables, theory notes, writing	P2 to P4	Every number in the draft traceable to a CSV; reproduction script runs from scratch

Part 2. Change specifications for the current implementation

Work packages (WP) are grouped by theme. Each states the goal, the files to touch, the exact change, the acceptance tests, the commands to run, pitfalls, and a ready prompt. Unless stated otherwise, paths are relative to the repository root and new modules live in the canary_ssm package. Names of existing functions are exact; names of new functions are proposals and may be adjusted if documented.

Group	Work packages	Theme
A	A1-A6	Protocol, configuration, manifest, seeds, experiment hygiene
B	B1-B6	Hyperparameter grids, selection objective, ablation fairness, frozen-model guard
C	C1-C6	Calibration methodology: regimes, sample size, exact validity test, transfer, recalibration
D	D1-D9	Model and estimator improvements (slope shrinkage, parameter-uncertainty SSM, support rule, robust likelihood, structure-adaptive detector)
E	E1-E3	Baselines and reference detectors
F	F1-F9	Metrics, statistics and reporting
G	G1-G8	Simulator realism
I	I1-I6	Engineering: reproducibility, parallelism, caching, tests, storage

Group A. Protocol, configuration, manifest, seeds

WP-A1. Separate the plain and step regimes (fixes I-01)

Goal. No calibration, validation or tuning mixture may contain step nulls unless the evaluation regime is the step regime. The step is a regime with its own calibration, not a family inside the main mixture.

Files. configs/default.yaml (rename), new configs/plain_main.yaml, new configs/step_regime.yaml, canary_ssm/experiments.py (generate_role, calibrate, evaluate, write_manifest), canary_ssm/report.py.

1. Create configs/plain_main.yaml from configs/default.yaml with the family S1_step removed and the remaining weights renormalised (S1_sparse 0.24, S2_class_wide 0.24, S3_broad 0.12, S4_weak_sparse 0.12, S5_label_errors 0.12, single_canary 0.12, transient 0.04). Add a top-level key `regime: plain`. Keep the old file as configs/legacy_mixture_with_step.yaml and add a comment that it must not be used for calibration.
2. Create configs/step_regime.yaml with the same families, each with overrides workload_step: 0.3365 added, and `regime: step`.
3. In generate_role, write the config's regime into data/<role>/manifest.json. In calibrate, copy it into policy.json as `regime`. In evaluate and validate_null, read the data manifest's regime and raise ValueError if it differs from the policy's regime, unless the CLI flag --allow-regime-mismatch is given (used only by audit-null and robustness studies; the flag value is recorded in the manifest).
4. In report.py, print the regime at the top of report.md and in gate_status.json.

Tests. test_regime_mismatch_refused (policy plain, data step, expect ValueError); test_regime_recorded_in_manifest; test_plain_main_has_no_step_overrides (parses the YAML and asserts no family has workload_step ≠ 0).

Commands.

```
python -m canary_ssm.cli simulate --config configs/plain_main.yaml --role dev --out data/plain/dev
python -m canary_ssm.cli simulate --config configs/step_regime.yaml --role dev --out data/step/dev
# repeat for calibration, null_validation, test under data/plain and data/step
```

Pitfall. The pilot's per-family calibration (P3) was criticised as not deployable because the family must be known at run time. Here regimes are calibrated separately for analysis; selecting the regime at run time is a distinct problem handled by WP-D4 and experiment R1.

Antigravity prompt
Implement WP-A1 of docs/Canary_SSM_Change_Plan.pdf exactly as written. Do not change any statistical code. Add the three tests. Run the full test suite and report. Do not start another work package.

WP-A2. Per-family validation sized for a valid gate (fixes I-02, replaces my earlier gate)

Goal. Make the calibration gate evaluable and statistically correct. Two corrections to my earlier design are required.

- **Per-family cells need real sample sizes.** The final run had 26 to 108 nulls per family, so no cell was evaluable by construction. Validation must generate a fixed number of nulls *per family*.
- **The gate 'upper bound at most 8%' is too strict for B = n = 500.** Under exact exchangeability the validated alert count follows a beta-binomial distribution (derivation in WP-C3). I computed that a perfectly valid method then fails the gate with probability 28.6%. With B = n = 2000 that falls to 0.6%. So the final validation must use at least 2000 calibration nulls and at least 2000 validation nulls per cell, or use the exact beta-binomial test of WP-C3 as the primary criterion.

Files. experiments.py (generate_role, validate_null), report.py (null_rate_table, calibration_gate), configs.

1. Extend the roles block of every config with an optional key per_family. Example: `null_validation: {per_family: 2000}`. In generate_role, when per_family is present generate that many nulls for every family (seeds from the registry, WP-A4, with family index in the seed formula). Record per-family counts in the data manifest.
2. In validate_null, add a table table_validation_by_family.csv with columns family, method, alerts, n, rate, ci_lo, ci_hi, plus the beta-binomial columns from WP-C3 (bb_pvalue_two_sided, expected_alerts).
3. In report.calibration_gate, take its input from the validation table (not from the test mixture). Replace MIN_N_GATE=400 by a parameter min_n=2000 for the final gate and keep 400 as an explicit 'exploratory gate' mode. Return status in {PASS, FAIL, NOT_EVALUABLE}. Never PASS with zero evaluable cells.
4. Budget: 8 families × 2000 nulls = 16000 rollouts. With the parallel runner of WP-I2 this is minutes to tens of minutes on 12 threads; without it, hours. Do WP-I2 first if running at this size.

Tests. test_gate_not_evaluable_when_n_small; test_gate_fail_when_interval_above_target; test_gate_pass_requires_evaluable_cell; test_per_family_counts_exact (generate 3 families × 7 nulls, assert counts).

Antigravity prompt
Implement WP-A2. First add the tests, confirm they fail, then implement. Show the new gate output for a toy run with 3 families and 50 nulls each (expect NOT_EVALUABLE) and for a synthetic table with n=2000 (expect PASS or FAIL by the numbers).

WP-A3. Complete the manifest (fixes I-10)

Goal. Any result file can be traced to code, data, seeds, parameters and environment.

Files. experiments.write_manifest, experiments.fit, new canary_ssm/manifest.py.

1. Run `git init` in the repository, commit the current state with message 'baseline after first full run', tag it v0.1-first-full-run. Every later work package merges on a branch and is tagged when it changes statistical behaviour.
2. Create manifest.py with `build_manifest(**extra) -> dict` collecting: git commit (via subprocess, record 'dirty' if uncommitted changes), config file content and SHA-256, regime, beta_mode, noise mode, factor rank, grid file and SHA-256 (WP-B1), candidate table SHA-256 (dev_selection.csv), selected hyperparameters, edge flags (WP-B2), seed registry version and the exact seed ranges used, package versions (numpy, scipy, pandas, statsmodels), Python version, OS, CPU model and thread count, wall time per stage, number of fit failures, and the model_hash and policy hash chain.
3. Make fit write model_config.json including `kind: tuned|frozen`, `beta_mode`, `noise`, `factor_rank`, `grid_version`. Make calibrate write policy.json including regime, B, alpha list, and the sorted maxima (full precision) for every method.
4. Create requirements.lock with pip freeze from the working environment (or a uv/pip-tools lock) and record its hash in every manifest.

Tests. test_manifest_has_required_keys (parametrised over the key list); test_manifest_records_dirty_flag (modify a tracked file in a temp repo).

WP-A4. Central seed registry (fixes I-14)

Goal. Disjoint seed ranges by construction. Create canary_ssm/seeds.py.

```
BLOCK = 1_000_000          # one block per purpose
FAMILY_STRIDE = 20_000     # up to 50 families per block, up to 20,000 rollouts per family
RANGES = {
  'smoke': 10_000, 'dev': 1_000_000, 'calibration': 2_000_000, 'null_validation': 3_000_000,
  'test': 4_000_000, 'stress': 5_000_000, 'audit': 6_000_000, 'noise_sweep': 7_000_000,
  'study_R1': 8_000_000, 'study_R2': 9_000_000, 'study_R3': 10_000_000, 'study_R4': 11_000_000,
  'study_R5': 12_000_000, 'study_R6': 13_000_000, 'study_R7': 14_000_000, 'study_R8': 15_000_000,
  'study_R9': 16_000_000, 'study_E3': 17_000_000, 'study_X': 18_000_000, 'realtrace': 20_000_000,
}
def seed_for(purpose: str, i: int, family_idx: int = 0) -> int: ...
def assert_disjoint() -> None: ...   # raises if two (purpose, family, i) ranges overlap
```

Rules: i must be below FAMILY_STRIDE, family_idx below 50; seed_for raises otherwise. Replace every ad-hoc seed in experiments.py (SEED_BASE dictionary, generate_role, audit_null) and in both experiment scripts. Keep a LEGACY dictionary documenting the seeds used by the first full run (dev 1,000,000+, calibration 2,000,000+, validation 3,000,000+, test 4,000,000+, Experiment 7: 2,100,000, 2,600,000, 7,500,000, 8,000,000, 8,200,000, 8,500,000; Experiment 8: 1,000,000, 2,000,000, 4,100,000, 6,300,000, 6,400,000, 7,300,000, 7,400,000) so old runs can be reproduced. Note that the legacy Experiment 8 calibration set (2,000,000+) coincided with the first full run's calibration seeds; new runs must not reuse them.

Tests. test_seed_ranges_disjoint (calls assert_disjoint); test_seed_for_bounds; test_no_adhoc_seed_literals (greps experiments.py and the studies package for integer literals of seven or more digits and fails unless the line contains seed_for or LEGACY).

WP-A5. Use the pipeline's eligibility everywhere (fixes I-13)

Experiment 8 computed release maxima with `r.decision_obs.all(axis=1)` as the eligibility mask. In the plain simulator without missingness this equals the pipeline's mask, but it is a different object and breaks as soon as missingness or late arrivals are active. Every study must obtain eligibility from DetectorOutput.elig (set by factor.adjust: at least 80% of canaries observed, at least 3 fresh controls per represented class, factor estimable). Add a lint test that greps canary_ssm/studies for decision_obs.all and fails.

WP-A6. Move experiment scripts into the package (fixes I-05)

Goal. Experiments become tested, parameterised, seed-registered commands that always use a tuned model and write standard outputs.

1. Create package canary_ssm/studies with one module per study (study_regime.py for Experiment 7, study_robust.py for Experiment 8, and later study_R1.py to study_R9.py, study_E1.py to study_E8.py). Each exposes `run(model_dir, policy_dir, out_dir, n_null, n_harm, seed_purpose, **kw) -> dict` and returns the manifest.
2. Add a CLI subcommand `study <name> --model ... --policy ... --out ... [--n-null N --n-harm M]` that dispatches by name. Remove sys.path hacks and hard-coded paths such as data_audit/model_pooled_dev.
3. Add `require_tuned(model_cfg)` raising unless model_config['kind'] == 'tuned' and `require_regime(policy, expected)`. Studies that deliberately use a frozen model (audits) must pass allow_frozen=True and record it.
4. Studies write only CSV and JSON plus figures generated from those CSV files by report functions. No study writes prose numbers. The Experiment 7 and 8 markdown reports are regenerated from CSV by report code.
5. Port the existing two scripts faithfully first (same logic, new seeds) and verify that with the legacy seeds they reproduce Antigravity's CSV tables within Monte Carlo noise; then apply the fixes from WP-D4 and WP-C5.

Tests. test_study_refuses_frozen_model; test_study_cli_dispatch; test_experiment7_legacy_reproduction (marked slow; run with legacy seeds and n=50; compares counts to a stored fixture).

Group B. Grids, selection objective, ablation fairness

WP-B1. Declared grids as data, extended and budget-matched (fixes I-03, I-15 partly)

Goal. Remove edge selections by extending grids, and make tuning budgets comparable across method families.

Files. new configs/grids_v2.yaml; experiments.candidate_grid, ssm_grid; baselines.config_grid.

1. Move grids out of code into configs/grids_v2.yaml with an explicit version string. candidate_grid(method, grids) reads them; the grid file hash is recorded in the manifest (WP-A3).
2. SSM grid v2 (applies to pooled, independent and hierarchical): rho in {0.9, 0.95, 0.99, 0.995, 0.999}; effect innovation SD sd_g in {0.0005, 0.001, 0.002, 0.005}; prior SD in {0.01, 0.02, 0.03}; local-to-class SD ratio sd_h_ratio fixed at 1 (a separate sensitivity study varies it over {0.5, 1, 2}). 60 raw combinations. Optional constraint long-run SD ≤ κ (see below).
3. Adjusted CUSUM: k in {0.1, 0.25, 0.5, 0.75, 1.0, 1.5} (6). Adjusted EWMA: alpha in {0.02, 0.05, 0.1, 0.2, 0.35, 0.5} (6). Top-K CUSUM: K in {1, 2, 3, 5, 7, 10} times k in {0.25, 0.5, 0.75, 1.0} (24). Matched contrast: window length w in {1, 5, 10, 15, 30} minutes of trailing mean (5). New baselines from WP-E1 get at most 24 candidates each.
4. Budget parity: report, for every method family, the number of candidates evaluated. If SSM families get more candidates than baselines, run the budget-parity study E3 (random search with equal numbers of candidates, repeated over random subsets) before claiming a ranking.
5. Long-run SD rule: the main grid keeps the constraint effect-state long-run SD ≤ κ (this removes candidates such as rho=0.999 with sd 0.001, long-run SD 0.022). Run an *unconstrained* variant as a sensitivity analysis in E3 and report whether the selected candidate or the ranking changes.
6. ρ=0.999 approximates a random-walk effect; the effective memory is 1/(1−ρ) = 1000 minutes, far longer than the 120-minute horizon, so the effect state is nearly a constant level. Record this interpretation in the paper: high persistence means 'effect is a level shift', which is the generative truth in the simulator (constant-size step). It is not evidence of anything on real data.

Tests. test_grid_file_hash_changes_with_edit; test_grid_counts_match_declared; test_constraint_removes_expected_candidates (hand-computed list).

WP-B2. Automatic edge-of-grid detection and grid expansion protocol (fixes I-03)

1. After select_on_dev, compute for each hyperparameter dimension whether the selected value equals the minimum or maximum of the values present in the grid for that method. Write edge_selection.csv (method, parameter, selected, grid_min, grid_max, at_edge) and add edge_flags to model_config.json.
2. The CLI prints a loud WARNING for any at_edge=True and exits with code 3 when --strict-edges is given.
3. Protocol: if any primary method is at an edge, create grids_v(N+1) extending that dimension by one step in the direction of the edge (geometric for scale parameters, e.g. k × 1.5, alpha × 0.5; for rho move toward 1 as 1−rho halves), rerun fit on the same development data, and record both grid versions and both candidate tables. At most two expansions; then freeze. Selecting a boundary value after two expansions is allowed but reported as 'boundary optimum' in the paper.
4. Never extend a grid using calibration, validation or test results.

Tests. test_edge_detection_on_synthetic_candidate_table (known optimum at edge and in interior).

WP-B3. Cross-validated selection with a one-standard-error rule (improves select_on_dev)

Current behaviour. select_on_dev uses a two-fold split by pair id: the threshold comes from nulls in one fold and capped time from harmed rollouts in the other. This is noisy and gives no uncertainty.

1. Replace by K-fold (default K = 5) with folds by `pair_id % K`. For each candidate store, once, the release maximum of every null and the score and eligibility arrays of every harmed rollout (as now). For fold f: threshold from the nulls of the other K−1 folds (B_train about 0.8 B_dev; k = ceil((B_train+1)(1−α))), capped time on harmed rollouts of fold f. Objective = mean over folds; also record the standard error across folds.
2. Selection rule: let J* be the minimum objective. Among candidates with objective ≤ J* + 1·SE(J*), choose the one with the smallest long-run SD of the effect state (SSMs) or the largest k/alpha regularisation (baselines). Record both the argmin and the rule-selected candidate; the rule-selected one is used.

3. Objective weighting option `objective: mixture | family_balanced`. Mixture weights harmed rollouts by the scenario mixture. family_balanced averages per-family means with equal weights. Use family_balanced for the final model if the paper reports per-family results.
4. Development nulls: with 300 dev pairs there are 300 nulls; 5 folds gives 240 training nulls and k = ceil(241 × 0.95) = 229. If development nulls are below 200, refuse (threshold too coarse). Prefer 500 dev pairs.

Tests. test_kfold_threshold_uses_only_training_nulls (assert via spies that held-out nulls are not in the maxima used); test_one_se_rule_picks_simpler; test_objective_weighting.

WP-B4. Fair no-persistence ablation (fixes I-04)

Problem. With rho=0 the effect state is white noise with SD sd_g. The grid caps sd_g at 0.01, so the state cannot represent the 0.0296 effect. The result 'persistence helps by 47 minutes' therefore partly measures a handicap. The same applies to the pilot's contrast A1.

1. Define hier_no_persist with its own grid: rho = 0; sd_g in {0.005, 0.01, 0.02, 0.03, 0.05}; prior SD in {0.01, 0.03, 0.05}; the long-run SD constraint does *not* apply (a white effect has long-run SD equal to sd_g by construction, and the purpose is to let it reach the effect size). 15 candidates.
2. Add hier_rw, a random-walk-effect variant: rho = 0.9999, sd_g in {0.0001, 0.0003, 0.001}, prior SD in {0.01, 0.02, 0.03} (9 candidates).
3. Add a stateless comparator: a windowed mean detector (WP-E1) that averages the adjusted residual of each host over the last w minutes. This is the non-state-space way to give the detector memory. If the windowed mean matches hier_no_persist, 'persistence' means nothing more than 'averaging over time'.
4. Report the ablation as a curve over rho in {0, 0.5, 0.8, 0.9, 0.95, 0.99, 0.999} with the other parameters re-tuned at each rho on development data. The headline statement 'persistence helps' is allowed only if the curve is monotone or has an interior optimum and the paired difference at the optimum versus rho=0 excludes zero.

WP-B5. Localization cutoff range and selection (fixes I-20 partly)

Extend the cutoff search from 1..5 to 1..n_canary (10 by default) and select on development data by mean F1 at landmark onset+30. Also report host AUC (area under the ROC curve of host scores at the landmark). Store the selected cutoff and the F1 curve in model_config.json and in the manifest.

WP-B6. Frozen versus tuned guard (fixes I-05)

model_config.json already records frozen. Add `kind` and make evaluate, every study, and report refuse a frozen model unless --allow-frozen is passed. A frozen model's outputs carry a banner 'FROZEN AUDIT MODEL, NOT FOR PERFORMANCE CLAIMS' in report.md. Test: test_evaluate_refuses_frozen.

Group C. Calibration methodology

WP-C1. Policy schema v2 with regimes

policy.json becomes `{schema: 2, model_hash, regimes: {plain: {B, alpha, regime, thresholds, maxima, advisory}, step: {...}}, default_regime: plain}`. Calibration of a regime stores the full-precision sorted maxima so that thresholds for any alpha (WP-C2), threshold uncertainty, and the exact validity test (WP-C3) can be computed later without re-running detectors. Keep reading schema 1 for the first-run artifacts.

WP-C2. Sample size, multiple alphas and threshold uncertainty (fixes I-18)

Why. The threshold is an order statistic of B maxima. Its own sampling variability makes the *achieved* false-alert rate of one calibrated policy random: it is Beta(B+1−k, k)-distributed (derivation in WP-C3). For B=500, alpha=0.05 the 95% range of the policy's true false-alert rate is 3.3% to 7.1% even when everything is exchangeable. That is a property of the sample size, not of the detector.

1. Final calibration uses B at least 2000 nulls per regime (range of the policy's true rate then about 4.0% to 6.0%). Development and exploratory work may use 500.

2. Add `thresholds_for_alpha(maxima, alphas)` to decision.py returning thresholds for alpha in {0.01, 0.02, 0.05, 0.10}. k=ceil((B+1)(1−alpha)); infinity if k exceeds B (for alpha=0.01 this needs B at least 99).
3. Add `threshold_uncertainty(maxima, alpha, n_boot=2000, seed)` returning the bootstrap 95% interval of the k-th order statistic and the exact Beta quantiles for the policy's true false-alert rate.
4. calibrate stores all four thresholds per method and the uncertainty summary in the policy.

Tests. test_thresholds_for_alpha_hand_values (B=500: k=476 for 0.05, 496 for 0.01, 451 for 0.10); test_beta_quantiles_match_simulation (simulate exchangeable Gaussian maxima 20,000 times, compare empirical quantiles of exceedance probability to Beta(B+1−k,k) within Monte Carlo error).

WP-C3. Exact validity test replacing the ad-hoc gate (fixes I-07 diagnosis)

Derivation. Let M_1..M_B be calibration maxima and M a new null maximum, all exchangeable and continuous. The threshold T = M_(k) (k-th smallest). Then F(T) ~ Beta(k, B+1−k), so the exceedance probability p = 1−F(T) ~ Beta(B+1−k, k). Given p, the number of alerts K among n independent validation nulls is Binomial(n, p). Hence **K ~ BetaBinomial(n, B+1−k, k)** exactly, with mean n(B+1−k)/(B+1). For B=500, k=476, n=500: mean 24.95 alerts, standard deviation 6.9 alerts. Observing 45 alerts (9.0%) has one-sided probability 0.006; observing 42 (8.4%, the pilot's M5 result) has 0.015.

1. Implement `betabinom_validity(k_alerts, n_val, B, k_order)` in metrics.py returning the expected alerts, the standard deviation, and the two-sided p-value (twice the smaller of the two one-sided tails, capped at 1) using scipy.stats.betabinom.
2. Gate G2 (replaces the CP-based gate as the *primary* validity criterion): a method passes a cell if the two-sided beta-binomial p-value is at least 0.01 *and* the achieved rate is at most 0.075. Keep the Clopper-Pearson interval in the tables for readers. Because the policy's own sampling variability is already inside the beta-binomial, this test is valid for any B and n and does not need the 400-null rule.
3. Multiplicity: the gate is evaluated on many cells (methods × families). Report the number of cells failing at p less than 0.01 and compare with the expected number under exchangeability (0.01 × cells). Fail the gate only if the failures exceed the expected count by a Bonferroni-style bound or if any failing cell is also above 0.075.
4. Exchangeability audit: for every method, compare the calibration maxima and the validation maxima with a two-sample Kolmogorov-Smirnov test and a Mann-Whitney test, and plot both empirical CDFs. A rejection means the calibration and validation data are not exchangeable (different generators, seeds drawn from different regimes, nuisance fitted on one of them) and is a bug until proven otherwise.

Tests. test_betabinom_mean_sd_hand_values; test_validity_p_uniform_under_exchangeability (repeat 2000 times: calibrate on B Gaussian maxima, validate on n; the p-values should be approximately uniform, KS p above 0.01); test_validity_detects_shift (shift validation maxima by 0.3 SD; p should be below 0.01).

Use on the first-run anomalies. Re-test Experiment 7's dual-regime step cells with this function: CUSUM 45 of 500 (p≈0.006 one-sided), top-K 38 of 500, hierarchical 34 of 500 (expected 25 ± 6.9). The CUSUM cell is probably a genuine mild violation; the hierarchical cell is not. See X-0.2.

WP-C4. Horizon consistency

The threshold must be calibrated over exactly the horizon in which alarms are allowed. release_max takes the maximum over eligible minutes from t0 to the end of the rollout, which is consistent with the 120-minute evaluation. If a study uses a shorter horizon (30 or 60 minutes), calibrate separately for that horizon; a threshold calibrated for 120 minutes is conservative for 30 minutes (the maximum over fewer minutes is smaller). Add horizon as a parameter to calibrate, store thresholds per horizon in {30, 60, 120}, and report power versus horizon. For rollouts with unequal follow-up (real traces), see WP-F3.

WP-C5. Recalibration command and calibration-transfer measurement (fixes I-08)

Goal. Separate two failure modes: (a) the filter is wrong; (b) the threshold was calibrated on data that does not look like the deployment environment. The pilot and Experiment 8 mixed them. Calibration transfer is measured by calibrating in one environment and validating in another.

1. Add CLI command `recalibrate --model M --data NEW_NULLS --regime NAME --out POLICY2`. It keeps the tuned model, optionally refits nuisance parameters on the new healthy data (flag --refit-nuisance), and

recomputes thresholds from the new nulls only. The new policy records the original model hash and the environment label.

2. Study R2 (Part 3) uses it: for each misspecification level, compare (i) nominal calibration; (ii) recalibration on matched nulls; (iii) nuisance refit on matched nulls with thresholds from matched nulls; (iv) nuisance estimated from controls (WP-D2) with nominal thresholds.
3. Interpretation rule: if matched recalibration restores the false-alert rate to the valid range, the lesson is that thresholds must be calibrated on the target environment's own healthy history, not that the filter is broken. If it does not, the filter's statistic is not exchangeable under that environment and needs a model fix.

WP-C6. Small-sample recalibration (how little healthy history is enough)

In production, healthy history is limited. Study the false-alert rate and its variability when thresholds are built from B in {50, 100, 200, 500, 1000, 2000} nulls. Provide an exact helper `min_B_for_alpha(alpha)` (the smallest B with k at most B, which is $B \geq (1-\alpha)/\alpha$; for $\alpha=0.05$ that is B=19, giving k=ceil(20×0.95)=19) and a conservative option that uses the (k+m)-th order statistic with m chosen so that the 95th percentile of the policy's true false-alert rate is at most alpha × 1.5. Report the power cost.

Group D. Model and estimator improvements

These work packages change statistical behaviour. Each is an *option* behind a flag, evaluated against the current default in a study before it can become the default. None may be adopted because it looks better on the benchmark mixture; adoption requires the paired contrast and the calibration gate on the pre-registered scenarios (Part 3).

Common notation. n canaries, N hosts, T = t0 + H time steps, t0 = 180, H = 120. y_{it} is log latency. $u_t = x_t$ − mean of x over the pre-period (centred workload). $\bar{f}_t$ is the control-derived factor estimate with estimation variance U_t. For host i: μ_i (intercept), β_i (workload slope), λ_i (factor loading). r_{it} is the adjusted residual passed to detectors. a_{it} is the effect. θ is the effect state vector, L the n × d effect map, e_i the AR(1) noise state with coefficient φ_i and innovation SD q_i.

WP-D1. Empirical-Bayes slope and loading shrinkage using the controls (fixes I-22, I-23)

Idea. The 90 control hosts give, before and after the rollout, a cross-sectional distribution of host slopes. A canary's own pre-period slope estimate is noisy (standard error about 0.02 with 180 minutes); the pooled development slope is unbiased only if hosts respond alike. Shrink each canary's slope toward its class's control mean with a weight that depends on how heterogeneous the controls actually are. This is the beta_mode value 'eb_controls'.

Algorithm. For every host h in the pre-period fit $y_{ht} = \mu_h + \beta_h u_t + \varepsilon_{ht}$ by least squares on observed, usable minutes. Let $\hat{\sigma}_h^2$ be the residual variance and $\hat{\phi}$ the AR coefficient from the nuisance fit (class-specific). Inflate for autocorrelation with $c = (1+\hat{\phi})/(1-\hat{\phi})$. Then

```
s_h^2   = c * sigma_h^2 / sum_t u_t^2          # sampling variance of the slope estimate
se_mu^2 = c * sigma_h^2 / T_obs                # sampling variance of the intercept estimate (u centred)
For class g with controls C_g (need |C_g| >= 5, else pool all controls):
  beta_bar_g = sum_j b_j / s_j^2 / sum_j 1/s_j^2          # precision-weighted mean of control slopes
  tau_g^2    = max( var_j(b_j) - mean_j(s_j^2), tau_floor^2 )   # between-host slope variance (method of moments)
For canary i in class g:
  w_i        = tau_g^2 / (tau_g^2 + s_i^2)
  beta_tilde = w_i * b_i + (1 - w_i) * beta_bar_g
  var_post   = w_i * s_i^2                                  # posterior variance of the slope
  mu_tilde   = mean_t( y_it - beta_tilde * u_t - lam_i * fhat_t )   # intercept re-fitted given beta_tilde
tau_floor = 0.005 (declared constant; sensitivity {0.002, 0.005, 0.01} in R1).
```

The same shrinkage applies to the factor loading λ_i using the control loadings (class mean and between-host variance). Keep per-host intercepts unshrunk.

Files. factor.py: add argument beta_mode to adjust(...) (values 'pre', 'pooled_dev', 'eb_controls'); extend the Adjusted dataclass with beta_tilde, beta_post_var, mu_post_var, lam_post_var (needed by D3); preprocess.Nuisance.beta_mode accepts the new value; fit and adjust_roll pass it through; CLI --beta-mode accepts it.

Tests. (1) test_eb_matches_pooled_when_tau_floor_dominates (set tau huge floor, expect β ≈ class mean); (2) test_eb_matches_per_host_when_tau_large; (3) test_eb_beta_rms_error_lower (simulate 40 rollouts with beta_jitter=0.08 and a step; RMS of β̂−β_true below both per-host and pooled; this is the Experiment 8 diagnostic reproduced with the new mode); (4) test_eb_future_perturbation (post-rollout data do not change β̂); (5) test_eb_singleton_class_fallback.

Success criterion for adoption. In study R1 the per-rollout post-minus-pre residual shift SD under a log(1.4) step with heterogeneous slopes (jitter 0.08) is below that of per-host pre-period slopes (Experiment 8 value 0.0152) by a bootstrap interval excluding zero, and calibration transfer from plain to step nulls improves (WP-C3 validity test).

Antigravity prompt
Implement WP-D1 (beta_mode eb_controls) exactly as specified, including the five tests. Do not change the default beta_mode. Then run only the unit tests and a 40-rollout diagnostic that prints SD and RMS of the per-rollout residual shift for per_host_pre, pooled_dev and eb_controls under a log(1.4) step with beta_jitter 0.08; save it as results/wp_d1/diagnostic.csv. Stop.

WP-D2. Noise parameters from the controls: aim for a pivotal statistic (fixes I-08, I-11)

Idea. The AR coefficient and innovation variance matter for calibration: an environment with more persistent noise than the development data (φe = 0.8 versus 0.6) raised false alerts to 24% to 35%. Control hosts are never exposed, so their residual series are healthy before and after the rollout. Estimating the noise parameters from the current rollout's controls makes the detector's null distribution depend less on the development environment. The goal is a *pivotal* statistic: one whose null distribution is the same across noise environments, so thresholds transfer.

1. In preprocess.py add `fit_noise_from_controls(adj, t_end, classes) -> (phi[g], q[g], v[g])`. It takes the control residuals c_jt (workload- and factor-adjusted, controls only) for t below t_end and maximises the same AR(1) innovation likelihood as fit_ar_class (scalar_nll, white SD pinned for ar_only). Use controls of class g for class g; pool if fewer than 5.
2. Mode 'controls_pre': use t_end = t0 (pre-period only). Mode 'controls_online': refit every 30 minutes with t_end = t, warm-started from the previous fit, parameters clipped to ±0.1 of the pre-period values to prevent instability; the filter state is kept (only φ, q change). Mode 'canaries_pre': use the canaries' own pre-period residuals (comparison arm).
3. EffectDetector accepts a per-rollout nuisance override: `EffectDetector(nuis, params, noise_source='dev'|'controls_pre'|'controls_online'|'canaries_pre')`. For 'dev' behaviour is unchanged.
4. Report the estimated φ and q for every rollout in a CSV so that estimation error versus truth (known in simulation) is available: columns rollout_id, class, phi_hat, q_hat, phi_true, q_true. Truth enters only the evaluator side.

Caveats to state in the paper. (i) The method assumes canary and control noise processes coincide before the rollout. A regression that changes noise variance is itself a (different) harm; D2 will not detect it and may absorb it online. This is why 'controls_online' needs the clipping and is evaluated separately. (ii) Using controls' post-rollout data to estimate noise does not leak truth but does make the statistic depend on control health; contamination of controls (R3) must be tested.

Tests. test_phi_recovery_from_controls (true φ in {0.3, 0.6, 0.8, 0.9}, error below 0.05 with 90 controls); test_controls_pre_causal; test_online_clipping; test_no_truth_import (extended lint).

Success criterion. Study R2: with nominal-environment thresholds and φe=0.8 data, the validated false-alert rate under 'controls_pre' falls inside the valid range of the WP-C3 test; if it does not, the statistic is not pivotal in φ and the paper must say that thresholds must be recalibrated per environment.

WP-D3. Parameter-uncertainty-aware filter (regression-state SSM, RS-SSM) (fixes I-22)

Problem. The current pipeline subtracts plug-in estimates μ̂_i + β̂_i u_t + λ̂_i f̂_t and then treats the residual as if the estimates were exact. Their errors become persistent offsets (a slope error times a sustained workload step; a loading error times a persistent factor shift; an intercept error) that the effect state mistakes for harm.

This is the mechanism behind the step failure.

Idea. Keep the errors in the state, with priors equal to the estimation posterior, so that the filter attributes an offset to estimation error if that is more probable than harm. This is a standard Bayesian treatment (regression coefficients as constant states) applied to the monitoring window.

Model. For canary i, let δ_i = (μ_i − μ̃_i, β_i − β̃_i, λ_i − λ̃_i) be the coefficient errors with prior N(0, Σ_i). Σ_i is block-diagonal (intercept, slope, loading treated as independent after centring) with diagonal entries se_mu², var_post (WP-D1) and lam_post_var, each multiplied by an inflation factor κ_Σ in {1, 1.5, 2} (a hyperparameter selected on development data, reported). With r̃_it = y_it − μ̃_i − β̃_i u_t − λ̃_i f̃_t,

```
state s_t = [ theta_t (d) | e_t (n) | delta_1..delta_n (3n) ]
F = blockdiag( diag(rho), diag(phi), I_{3n} )          # coefficient errors are constant
Q = blockdiag( diag(sd^2), diag(q^2), 0 )
H_t row i = [ L_i , e_i^T , 0..0, (1, u_t, fhat_t) in block i, 0..0 ]     # TIME-VARYING design
V_t = lam lam' U_t   (white SD pinned)
P_0 = blockdiag( prior_sd^2 I_d , P_e(pre-period filtered) , blockdiag(Sigma_i) )
Run from t0 only. The pre-period fit supplies mu_tilde, beta_tilde, lam_tilde and Sigma_i; no pre-period filtering of delta is nee
```

State dimension d + 4n (d = n for the hierarchical model): 50 for n = 10 and 500 for n = 100. Per-update cost grows cubically; at n = 10 it is about the same order as now (sub-millisecond). For n at least 50 implement the observation update in information form exploiting the block structure, or restrict δ to slope only. Report runtime in study R7.

Files. new canary_ssm/rs_ssm.py with class RSEffectDetector (same interface as EffectDetector: run(adj) returning DetectorOutput); factor.Adjusted gains psi_tilde (nc,3), psi_cov (nc,3,3), u (T,), and f_hat already exists; kalman.KalmanFilter.update already accepts a per-call H_obs, so time-varying H needs no change.

Tests (all mandatory). (1) test_rs_reduces_to_ssm_when_sigma_zero: with Σ_i = 0 the scores equal EffectDetector's to 1e-9. (2) test_rs_matches_statsmodels_time_varying_design: a small system with time-varying design matrix compared to statsmodels (which supports time-varying design arrays). (3) test_rs_causal_future_perturbation. (4) test_rs_prior_explains_step_offset: construct r̃ = δ_slope u_t with u_t constant after t0 and no effect; the posterior mean of the effect must be smaller than in EffectDetector. (5) test_rs_still_detects_effect: with a clean 3% effect and no step, power at fixed threshold is within a stated tolerance of EffectDetector (not lower by more than X; X from a pilot of 100 rollouts, recorded in the test).

Identifiability statement for the paper. When the workload is constant after the step, an intercept error, a slope error times the constant u, and a level effect produce the same observation pattern; the filter separates them only through their priors. When workload keeps varying after the step, the slope error is identifiable from the covariance between residual and u_t. Quantify this in study R1 by varying post-step workload variance.

Antigravity prompt
Implement WP-D3 (rs_ssm.py) after WP-D1 is merged. Write the four mandatory tests first, including the statsmodels time-varying reference. Do not add the module to the default method list; register it as method name rs_hier_ssm behind a flag. Report the state dimension and per-update runtime for n=10, 50. Stop before any benchmark.

WP-D4. Support-based regime statistic replacing the simulator-tuned selector (fixes I-06)

Problem. Experiment 7's selector is (mean x over t0..t0+4) − (mean x over the 5 minutes before t0) > 0.17. The constant 0.17 is half of the simulated step 0.3365; the step always starts exactly at t0; and the selector uses minutes t0..t0+4 to choose the threshold applied from t0 on. Its accuracy (which is trivially near 100% in this simulator) was never saved. It demonstrates nothing about deployability.

Replacement. A scale-free, data-driven, strictly causal statistic. Let x̄_pre, s_pre, x_min, x_max be the pre-period mean, SD, minimum and maximum of the workload covariate. For each t at or after t0 define

```
S_out(t)   = max(0, x_t - x_max, x_min - x_t) / s_pre                # distance outside pre-period support, in SD units
S_shift(t) = | mean(x_{t-m+1..t}) - x_bar_pre | / s_pre   with m = 5     # trailing-window shift, past data only
stress_t   = 1[ max(S_out(t), S_shift(t)) > s0 ]                       # per-minute stress flag
s0 is set on DEVELOPMENT plain nulls as the 99.9th percentile of max_t max(S_out, S_shift) (so that a plain rollout rarely flags).
```

Policies. P0 ignore the flag (current behaviour). P1 *abstain*: no alarms while stress_t = 1 and for h further minutes (h in {0, 10, 30}, selected on development data). P2 *latched regime*: the regime is 'stressed' from the

first minute the flag is raised within a decision window of W minutes after t0 (W in {0, 5, 10}) and stays so; alarms are suppressed during the decision window for all rollouts (this makes P2 strictly causal at the cost of W minutes of delay, reported); thresholds come from the plain or step policy of WP-C1. P3 *conditional threshold by continuous stress*: threshold = plain threshold + γ × (running max of S) with γ chosen on development data; reported as an exploratory variant.

Validity. P1 and P2 are valid only if the abstention or regime decision is a function of the covariate x alone, which is independent of the latency noise under the null, so exchangeability within regime holds. State this in the paper and test it empirically (WP-C3 test) in both regimes. P3 does not have a finite-sample guarantee.

Files. new canary_ssm/support.py (statistics and policies); decision.first_alarm gains an optional mask argument `allowed: bool array`; experiments.evaluate and the studies pass the policy mask; policy.json regimes (WP-C1).

Tests. test_support_stat_causal (perturb future x, past S unchanged); test_s0_from_dev_only; test_p1_masks_alarms; test_p2_no_alarm_inside_window; test_plain_rollouts_rarely_flag (at most 1% on 1000 plain nulls).

Evaluation. Study R1: regime-detection ROC (TPR versus FPR as s0 varies) over step sizes {0.1, 0.2, 0.3365, 0.5, 0.7}, ramp lengths {5, 15, 30, 60} minutes, onset offsets {−30, −10, 0, +10, +30, +60} minutes relative to t0, spikes of {5, 15} minutes; then false-alert rates and power under P0 to P3. Report 'detection delay cost' (extra capped time) and 'abstention minutes'.

Antigravity prompt
Implement WP-D4 (support.py, masks in decision.first_alarm). Derive s0 from development plain nulls only. Replace the hard-coded 0.17 in the ported Experiment 7 study. Do not tune on step nulls. Report the ROC of the stress flag over the step sizes, ramps and onset offsets listed in the document, saved as CSV.

WP-D5. Robust observation handling for heavy tails (fixes part of I-11)

Current robustness is winsorisation at c = 3 pre-period SDs (hier_huber). In Experiment 8 it lowered Student-t(3) false alerts from 12.8% to 9.5% and cost 4.7 points of power; the test did not recalibrate on heavy-tailed nulls, so it measured calibration transfer, not robustness.

1. Add a robust Kalman update in kalman.py: `update_robust(r_obs, H, V, nu=4.0, iters=1)`. After the Gaussian innovation v and covariance S, compute per-observation standardised innovations z_k = v_k / sqrt(S_kk); weights w_k = (nu + 1) / (nu + z_k²); inflate V_kk by 1/w_k; recompute K and the update; iterate once. This is a scale-mixture (Student-t) approximation. Keep the Joseph form and symmetrisation.
2. Expose as method hier_t (nu in {3, 4, 6, 10}); the Huber clipping variant stays with c in {2, 2.5, 3, 4}. Tune nu or c on development data that include a heavy-tail family.
3. Calibrate robustly: every robust variant must be recalibrated on the same heavy-tailed environment it is evaluated in, and also evaluated with nominal thresholds (transfer). Report both.

Tests. test_robust_downweights_outlier (a 10-SD spike moves the posterior less than the Gaussian update); test_robust_equals_gaussian_for_large_nu (nu=1e6, tolerance 1e-6); test_robust_psd (covariance stays symmetric positive definite under extreme z).

WP-D6. Factor-shock sensitivity and per-host noise scale (fixes part of I-11)

- **Shock sensitivity.** The simulator's shared shock is log(1.05) = 0.0488 at t0 decaying at φf = 0.8. A larger shock multiplies the effect of loading error (λ_i − λ̂_i) f_t. Measure false-alert rate versus shock size {0, 0.05, 0.15, 0.3} and loading dispersion lam_host_sd {0.02, 0.05, 0.15, 0.3} (study R2). The mitigation is D3 (loading error states) and D1 (loading shrinkage).
- **Per-host noise scale.** Currently q_e and φe are class-level. Hosts with different noise levels (e.g. an older hardware class) get the wrong weight. Add per-host scale s_i = σ̂_i / mean class σ̂ from the pre-period residuals (clipped to [0.5, 2]) and use q_i = s_i q_class. Simulator support: WP-G5. Test: test_per_host_scale_improves_loglik_on_heteroskedastic_data.

WP-D7. Release score alternatives

The release score is the log of the maximum host risk. Evaluate alternatives under identical calibration, because the score shapes which alarms rank first and thus power at fixed false-alert rate:

Score	Definition	Cost / note
max-host risk (current)	log max_i Pr(a_i > κ)	Cheap; sensitive to a single host
union bound	log min(1, Σ_i Pr(a_i > κ))	Cheap; upper bound on Pr(any host harmful)
Monte Carlo any-host	fraction of N=4000 draws from N(L m, L P L') with max_i a_i > κ; report MC standard error; floor at 1/N and use the union bound when below	Fixed seed per rollout; label it an instantaneous model-conditional probability, not an event probability
posterior mean max	E[max_i a_i]	Not a probability; may rank well
class-aware	max over classes of Pr(class effect > κ), plus max over hosts of local risk	Targets class-wide harm

Tests. test_mc_matches_closed_form_single_host; test_union_bound_ge_max; test_mc_seeded_reproducible. Report the rank correlation of scores and the power-at-α curves (F4).

WP-D8. Hypothesis-enumeration Bayesian detector (targets the unknown affected subset; fixes I-12)

Idea. Sparse multi-stream change detection asks which subset of streams changed. The hierarchical model shares strength within a class and so underperforms for one or two affected hosts; independent hosts ignore structure. Enumerate a small, explicit set of hypotheses about *which hosts are affected* and let the data choose, accumulating the predictive likelihood of each hypothesis. This is a Bayesian analogue of a subset GLR.

Hypothesis set. H0: no effect. H_i: only canary i (n singletons). H_g: all canaries of class g (G classes). H_all: all canaries. For n = 10 and two classes that is 1 + 10 + 2 + 1 = 14 hypotheses. Optionally add pairs and triples with a prior penalty (sparse extension, exploratory).

```
For each hypothesis h with effect-map column L_h (n x 1 indicator of the affected set S_h), run a Kalman filter with
  state [theta_h (1), e (n)],  F = blockdiag(rho, diag(phi)),  Q = blockdiag(sd^2, diag(q^2)),
  H_obs = [L_h, I_n] on the observed hosts,  V_t = lam lam' U_t,  P_0 = blockdiag(prior_sd^2, P_e).
H0 uses the noise-only filter. Accumulate the predictive log-likelihood increment
  ell_{h,t} = log N( v_{h,t} ; 0, S_{h,t} )     for t >= t0.
Posterior over hypotheses (prior pi_h, fixed-share mixing eps in {0, 0.01}):
  w_{h,t}  proportional to  ( (1-eps) w_{h,t-1} + eps/K ) * exp( ell_{h,t} - ell_{0,t} )     # relative to H0
Host risk:    p_i,t = sum_{h : i in S_h} pi_{h,t} * Pr_h( theta_h > kappa | data <= t )
Release score: log max_i p_i,t     (or log of the probability of any harmful hypothesis)
Prior: pi_0 = 0.5; remaining mass split as (m_single, m_class, m_all) with m_single + m_class + m_all = 1, selected on development
```

Cost: K independent n-dimensional filters, about 14 times the current per-update cost (still of order millisecond at n = 10). Efficiency note: hosts outside S_h contribute the same likelihood under every hypothesis and cancel in ratios; an implementation that filters only the hosts in S_h is possible once the cross-host factor-error covariance is neglected, but the full-filter version is the reference and the optimisation must reproduce its scores within 1e-8.

Files. new canary_ssm/enum_ssm.py with class HypEnumDetector (interface as EffectDetector). **Tests.** test_enum_weights_concentrate_on_true_subset (large simulated effect on a known host; weight on H_i exceeds 0.9 after 30 minutes); test_enum_class_wide_prefers_class_hypothesis; test_enum_null_stays_near_H0; test_enum_causal; test_enum_matches_efficient_version.

WP-D9. Structure-averaged detector (cheaper cousin of D8)

Run the three existing structures (pooled, hierarchical, independent) in parallel on the same residuals; weight them by their running predictive likelihood (Bayesian model averaging, optionally with fixed-share mixing). Host risk is the weighted average of the three models' host risks. Compare with the jointly calibrated union detectors (combo_*) and with D8. Cost is three filters. Tests: test_bma_weights_follow_truth (class-wide effect favours pooled/hierarchical; single-host favours independent when the effect is large).

Hyperparameters. Prior weights (1/3, 1/3, 1/3); fixed-share eps in {0, 0.01, 0.05}. Each constituent keeps its development-selected (ρ, sd, prior). Report whether averaging recovers the best of the three in every scenario family: this is the claim D9 stands or falls on.

Antigravity prompt
Implement WP-D8 and WP-D9 in separate branches, each with its tests, after WP-A and WP-B are merged. They are options behind method names enum_ssm and bma_ssm. Do not benchmark them yet. Report per-update runtime at n=10.

Group E. Baselines and reference detectors

A reviewer's first question is whether the baselines are strong and fairly tuned. The first run had four baselines tuned on narrow grids, and no detector designed for the unknown affected subset or for always-valid sequential inference. All baselines below operate on the same adjusted residual stream r_it, with the same masks, eligibility and calibration as every other method. Each has at most 24 candidates, selected by the K-fold protocol of WP-B3 on development data, and each is calibrated on plain nulls only.

WP-E1. Additional baselines

Method (name)	Definition	Candidates	Notes
windowed_mean (wmean)	Per host, mean of the last w standardised residuals; release score = max over hosts (and a top-K variant)	w in {5, 10, 15, 30, 45}; K in {1, 3, 5}	Stateless memory; the fair comparator for 'persistence' (B4)
ar_cusum	CUSUM on AR(1)-prewhitened residuals: e_t = (z_t − φ z_{t−1}); a persistent shift δ in z appears as (1−φ)δ in e_t, so the reference value is scaled accordingly	k in {0.1, 0.25, 0.5, 0.75}, φ from nuisance	Standard residual-chart treatment of autocorrelation
glr_window	Per host sliding-window GLR for a mean shift: max over s in the last W minutes of (Σ_{j=s..t} z_j)² / (2 (t−s+1)); release = max over hosts (and top-K)	W in {15, 30, 45, 60}; K in {1, 3, 5}	Unknown change time and unknown shift size; prewhiten first
sparse_mix_glr (Xie-Siegmund)	Multi-stream mixture GLR for a change in an unknown subset of streams (Annals of Statistics 41(2), 670-692)	sparsity parameter p0 in {0.1, 0.3, 0.5}; window W in {30, 60}	Implement the statistic from the paper (not from memory); document the exact form; verify against a hand-computed small case; this is the baseline needed before any sparse-detection claim
bocpd (Adams-MacKay)	Bayesian online changepoint detection with a Gaussian model with unknown mean and variance (conjugate), constant hazard 1/λ_h; per host; score = max over hosts of Pr(run length shorter than t−t0+1 AND post-change mean positive)	λ_h in {60, 120, 240}	Use directional restriction; arXiv:0710.3742
msprt	Mixture sequential probability ratio test per host with a Gaussian mixing distribution N(0, τ²) on the mean shift: Λ_t = sqrt(σ²/(σ²+tτ²)) exp(τ² S_t² / (2σ²(σ²+tτ²))), S_t the running sum of prewhitened z; always-valid p-value p_t = min over s ≤ t of min(1, 1/Λ_s); score = −log p_t max over hosts	τ in {0.25, 0.5, 1.0, 2.0}	The 'peeking-safe' approach used in online experimentation; verify the formula and the always-valid construction against Johari, Pekelis, Walsh before citing
mw_judge	Rank-based canary judge in the spirit of Kayenta: within a sliding window of w minutes compare canary versus class-matched control residuals with a Mann-Whitney U statistic per class; score = max class z	w in {15, 30}	Cite Kayenta documentation after checking it; operationally relevant comparator

Files. new canary_ssm/baselines_extra.py; register in baselines.config_grid and run_baseline; add to experiments.BASELINES; reports include them automatically. **Tests (each):** a hand-computed fixture (e.g. for msprt with τ=1, σ=1 and three observations, compute Λ_t by hand and compare); a null-distribution sanity check (maxima finite and increasing in horizon); causality; missing-data invariance (a missing minute leaves state unchanged); a test that the method does not read truth.

Note. Do not copy statistics from memory. For Xie-Siegmund, BOCPD and mSPRT, the agent must read the cited source, write the statistic in docs/baselines.md with equation numbers, and implement exactly that. If a statistic cannot be reproduced from the source, stop and report; do not substitute a lookalike and call it by the paper's name.

WP-E2. Clairvoyant reference detectors (upper bounds, never competitors)

These use ground truth and live in canary_ssm/oracles.py, which is the only detector-like module allowed to import Truth. Their outputs are labelled ORACLE in every table and plot and are excluded from rankings. They show how much of the achievable power each real method captures.

- **oracle_cusum:** CUSUM on the standardised residual of the true affected hosts only, with reference value k = δ/(2σ_i), where δ is the true effect and σ_i the pre-period residual SD. Calibrated on nulls like any other method, using the same hosts set that is affected in the harmed rollout (for nulls use a random host subset of the same size so that the null distribution reflects the same statistic).
- **oracle_bayes:** the hierarchical or independent SSM run with the *true* φ, q and, for effect parameters, a prior matching the generative effect (a persistent level with the true size as prior mean 0 and SD equal to the effect size). It bounds what any SSM tuning could achieve.
- **oracle_subset:** the independent SSM restricted to the true affected set.

Report the efficiency ratio (real method capped time − oracle capped time) / (miss-everything capped time − oracle capped time) in [0, 1]. This normalises differences across scenarios.

WP-E3. Tuning-budget parity and fairness protocol

1. Every method family reports: number of candidates, grid version, hyperparameters selected, whether at an edge, development objective curve.
2. Run study E3 (Part 3): random search with M candidates per family, M in {6, 12, 24, 48, 96}, repeated 20 times with different random candidate subsets; plot best-so-far development objective and test capped time versus M. If the method ranking depends on M, state so.
3. State in the paper the development data size and the ratio of development to test data for every family. No method may have seen more development data than another.

Group F. Metrics, statistics and reporting

WP-F1. Pre-registration and the contrast registry

Create docs/PREREG.md (template in Appendix H) and a machine-readable configs/prereg.yaml listing every contrast, scenario set, noise levels, effect sizes, α, margin of practical equivalence, and the decision rule. Code in stats.py reads prereg.yaml and computes exactly those contrasts. Contrasts not in the file are labelled 'exploratory' in every output. Commit and tag the file before the test set is generated (tag prereg-v1).

Primary contrasts (proposed). C1 hier versus independent SSM; C2 hier versus pooled SSM; C3 best SSM versus best tuned classical baseline (selected on development data, not test); C4 persistence (hier versus fair no-persistence, B4); C5 factor adjustment (hier versus rank0); C6 AR noise (hier versus white); C7 label errors (S5 versus S2 for hier); C8 structure-adaptive (D8 or D9) versus the better of hier and independent. Each is evaluated in the plain regime, per scenario family, per noise level, and pooled over scenarios with scenario-balanced weights. A **margin of practical equivalence** of 3 minutes of capped time (declared in advance) separates 'no difference' from 'ties inside the margin'.

WP-F2. Paired inference with multiplicity control

1. Unit of resampling: the rollout pair (null and harmed share exogenous arrays; the harmed rollout is the analysis unit for capped time). Bootstrap 10,000 resamples of pair ids; percentile intervals; also BCa as a check. Never resample timestamps or hosts.
2. Paired differences across methods on the same harmed rollouts: d_p = capped_A(p) − capped_B(p). Report mean, 95% interval, and the standardised effect size d / SD(d).
3. Multiplicity: Holm-Bonferroni across the primary contrasts within a family of claims; report adjusted p-values from a paired permutation test (sign-flip of d_p) with 20,000 flips. Keep unadjusted intervals in the tables for transparency.
4. Equivalence: report whether the 95% interval lies inside ±margin (TOST logic), so 'tie' is a statistical statement.

5. Heterogeneity: report the contrast within each scenario and the between-scenario SD of contrasts; a pooled contrast hiding opposite signs is flagged automatically (the code compares the signs of per-scenario intervals).

Sample size. From the first run, the SD of the paired capped-time difference between hier and pooled is about 52 minutes (derived from the reported interval: half-width 4.67 at n = 474 gives SE 2.38 and SD about 52). The number of pairs needed for 80% power at two-sided 5% is $n \approx (2.8 \times SD / \delta)^2$:

True difference δ (minutes)	10	5	3	2
Pairs needed (SD = 52)	about 212	about 848	about 2354	about 5298

These are estimates; recompute with the SD from the fair benchmark. They explain why contrasts below about 3 minutes cannot be resolved with 500 pairs and why equivalence margins must be at least 3 minutes unless n is increased.

WP-F3. Time-to-detection analysis (survival view)

Horizon-capped time is the restricted mean survival time (RMST) of the time-to-success variable, with non-success treated as censored at the remaining horizon. Add stats.py functions `km_curve(times, events)` (Kaplan-Meier cumulative incidence of success with Greenwood intervals) and `rmst(times, events, tau)`. Show cumulative incidence of detection versus minutes since onset per method, with intervals. Premature alarms are competing events (they end the rollout's chance of a clean success): report cumulative incidence functions for success and for premature alarm separately. For rollouts with different follow-up (real traces) use the same code with censoring at each rollout's own end.

WP-F4. Power versus false-alert curves

Instead of a single α = 0.05, report each method at α in {0.01, 0.02, 0.05, 0.10, 0.20} using thresholds from the stored calibration maxima (WP-C2). For every α compute success probability at 15, 30, 60 minutes and capped time. Summaries: (i) power at 30 minutes versus α curves; (ii) the area under the power-versus-α curve over α in [0, 0.2] (a threshold-free summary of detection ability at calibrated rates); (iii) the minimum detectable effect (smallest effect size with 80% success at 30 minutes and α = 0.05) from study E4. Note that for α = 0.01 the validation needs at least about 2000 nulls to resolve the rate.

WP-F5. Host-level inference: localization, FDR and probability calibration

- Localization: top-3 recall, F1 at the dev-selected cutoff, AUC of host scores at onset+30 and at onset+60, and the full precision-recall curve.
- Bayesian FDR: for models with host probabilities, select hosts with $Pr(a_i > \kappa)$ above a threshold chosen so that the average of $(1 - p_i)$ among selected hosts is at most 0.1 (posterior expected false discovery proportion). Report the *achieved* false discovery proportion against truth; model-based FDR is not frequentist control and the paper must say so.
- Reliability: reliability curves of $Pr(a_{i,t} > \kappa)$ versus the frequency of $J(i,t)$ with Wilson intervals per bin (bins of equal count, 10 bins), Brier score with its decomposition (reliability, resolution, uncertainty), and log score. ECE is reported only with its binning stated and is never the headline number.
- Condition on the event being active (J = 1 somewhere) and unconditionally; probabilities are mostly near zero under nulls and unconditional reliability is dominated by them.

WP-F6. Decision-theoretic evaluation (operational loss)

Define a loss for the release decision: C_{miss} per affected-host-minute of undetected harm (harm-minutes), C_{fa} per false alert, C_{delay} folded into harm-minutes. Harm-minutes = Σ over affected canaries of the minutes between onset and alarm (or to horizon if missed), weighted by an assumed traffic share per canary (default equal). Expected loss = $C_{fa} \times P(\text{false alert}) + C_{miss} \times E[\text{harm-minutes}]$. Sweep the ratio $r = C_{fa} / C_{miss}$ over a log grid (0.1 to 1000) and plot expected loss and the decision curve (net benefit versus threshold). Give each method its own α chosen on development data to minimise development loss for each r (a threshold-free comparison of decision value). State that costs are assumptions; the paper presents the curve rather than one cost.

WP-F7. Alert flicker and advisory policy

The primary policy latches alarms. A reversible 'pause/promote' policy is operationally relevant: the existing advisory_replay (NORMAL to PAUSE at h_on; PAUSE to NORMAL after n_clear below h_off) and calibrate_advisory give reversals and minutes paused. Evaluate: reversals per null rollout, minutes paused per null rollout, time to first pause on harmed rollouts, and time paused after recovery, for h_off/h_on gaps hyst_frac in {0.25, 0.5, 0.75} and n_clear in {3, 5, 10}, selected on development data. Report as a secondary table; the first-pause rate is controlled by the same threshold as the latched policy.

WP-F8. Stratified and sensitivity reporting

Every headline number is accompanied, in an appendix generated by code, by its stratification over: effect size, noise SD, number of affected canaries, class split, scenario family, missingness level. Add automatic detection of Simpson-type reversals (a pooled contrast with the opposite sign to the majority of strata) and print them in report.md.

WP-F9. Report generator hardening

1. report.py writes `report.md` only from CSV; add the function assert_report_numbers(res_dir) that re-reads every number in report.md via a regular expression against the CSVs and fails on mismatch (the summary-integrity gate of the pilot).
2. Figures are generated by functions in report.py (detection curves, power-α curves, reliability, null-rate forest plot with CIs by family, abstention heatmap, paired-difference forest plot, calibration validity plot with the beta-binomial band). Each figure has a CSV twin saved beside it.
3. Add a 'claims table' output: a machine-checked list of the allowed and disallowed claims (Part 5.8) with evidence pointers, so that the paper's claims section is generated, not written.

Group G. Simulator realism

The simulator is the paper's biggest threat to validity (I-16). The extensions below are independent, each adds new SimConfig fields with defaults that reproduce the current simulator exactly (a regression test asserts bit-identical output for default configs), and each is evaluated as its own scenario family or ablation. Implement them in simulate.py with new named RNG streams appended to STREAMS so that existing streams are unchanged.

WP	Extension	New fields (proposed)	Why it matters
G1	Request-level latency: per minute, host i serves N_it ~ Poisson(share_i × traffic_t) requests with lognormal latency; the metric is log of the mean, p50, p90 or p99 of that minute's requests. Observation noise then scales with 1/N and is non-Gaussian for tail percentiles	metric: mean|p50|p90|p99; req_sigma; base_traffic	Real latency metrics are aggregates with heteroskedastic, non-Gaussian noise; tests the Gaussian filter
G2	Realistic workload: diurnal and weekly seasonality, ramps, spikes, autoscaling events (hosts join/leave), load-balancer weights that differ per host	diurnal_amp, weekly_amp, spike_prob, autoscale_events, share_sd	Host-specific traffic shares make workload slopes differ by host (the heterogeneity that broke pooled slopes)
G3	Shared incidents of varied shapes (pulse, ramp, plateau, double pulse) acting on canaries and controls with host-specific loadings; local harm plus incident mixtures	incident_shape, incident_len, incident_size, incident_loading_sd	The attribution problem: shared harm must not alarm
G4	Effect shapes: gradual ramp, intermittent (flapping), load-dependent (effect proportional to workload level), recovery, improvements (negative effects), sub-κ effects	shape in {abrupt, ramp, flap, load_dep}, flap_period, load_coef, negative_effect	Persistence is not always the truth; improvements must not alarm; sub-κ effects test the harm boundary
G5	Host heterogeneity: hardware classes with different μ, noise SD, φ, workload slopes, loadings; heteroskedastic hosts	class_noise_mult, class_phi, class_beta	Per-host noise scale (D6), class structure meaning
G6	Fleet persistence and long history: the same hosts across rollouts and a pre-period of 180, 360, 720 or 1440 minutes	t_pre in sweep; persistent_fleet seed	Slope identification scales with sqrt(T Var x); real systems have days of history

WP	Extension	New fields (proposed)	Why it matters
G7	Rollout structure: staged exposure (1% to 5% to 25%), varying canary counts (1 to 50), 3 or more classes, unbalanced classes, singleton classes	n_canary, n_classes, class_sizes, stages	Class structure assumptions of the hierarchy
G8	Data pathologies: late, out-of-order and duplicate records, clock skew between hosts, stuck (flatline) metrics, instrument glitches (single-sample spikes), informative missingness (dropout correlated with effect or load)	dup_prob, skew_minutes, stuck_prob, glitch_prob, mnar_coef	Operational realism; abstention behaviour

Tests for each extension: defaults reproduce legacy output exactly; the new feature changes only what it claims to change (paired null/harm differ only by the injection); ground truth stays in Truth; schema round trip through telemetry CSV preserves detector-visible fields. G1 additionally tests that the sampling variance of the minute metric equals the analytical value.

Group I. Engineering, reproducibility and speed

WP-I1. Version control, locking, task runner

1. git init, .gitignore (data, artifacts, results/final), commit, tag v0.1-first-full-run (WP-A3). Branch per work package; squash-merge after tests pass; tag when statistical behaviour changes.
2. requirements.lock from pip freeze; pyproject extras: dev (pytest, statsmodels, hypothesis), report (tabulate).
3. A top-level script tasks.ps1 (and tasks.sh) with named targets: test, smoke, plain-pipeline, step-pipeline, audit, studies, report. Each target prints the commands it runs.

WP-I2. Parallel runner, caching and resume (fixes I-17)

The first run's fit used one process. The PC has 12 hardware threads; leave two free.

1. New canary_ssm/parallel.py with `pmap(fn, items, workers, chunk, desc)` using concurrent.futures.ProcessPoolExecutor on Windows (spawn start method: functions must be top-level and picklable; no lambdas; pass paths, not large arrays). Deterministic: per-item seeds come from the registry, never from worker identity; results are returned in input order. Default workers = max(1, cpu_count − 2).
2. Parallelise over rollouts in calibrate, validate_null, audit_null, evaluate and over candidates in select_on_dev (candidate-major: each worker evaluates one candidate across all dev rollouts).
3. Cache adjusted residuals: cache/adj/<nuisance_hash>/<rollout_id>.npz holding Adjusted fields; reused by every method and candidate. Cache detector outputs for selection: cache/sel/<method>/<cand_hash>/<rollout_id>.npz holding score and eligibility only. Keys include the nuisance hash and code version so stale caches are never read.
4. Resume: long commands write a progress journal (JSON lines); on restart they skip completed chunks. Add --resume.
5. Numerical determinism test: the same command with workers=1 and workers=8 produces byte-identical CSV outputs (sorted by key).

Windows pitfalls. Guard entry points with `if __name__ == '__main__':`; set OMP_NUM_THREADS=1, MKL_NUM_THREADS=1 and OPENBLAS_NUM_THREADS=1 in the worker initialiser (oversubscription of BLAS threads otherwise slows 12 workers badly); avoid closing over large objects; use temporary files for large inter-process data.

WP-I3. Test-suite expansion

Area	New tests
Kalman	hypothesis-based property tests: covariance symmetric PSD after random update sequences; invariance to observation order within a step; equivalence of sequential scalar updates and the joint update for diagonal V
Calibration	validity p-value uniformity under exchangeability (WP-C3); threshold-for-alpha hand values; infinite threshold when B small

Area	New tests
Leakage	AST lint extended to every detector module, studies, and baselines_extra; oracles.py explicitly exempt and labelled
Regression	one test per bug ever found: kind='null' parsed as NaN; gate PASS with zero evaluable cells; selection fold too small; policy for different model hash
Simulator	legacy-default bit-identity; paired invariants for each new feature; truth never in Rollout
Reports	assert_report_numbers on a fixture; figure generation smoke test; every PASS/FAIL string traceable to a number
Determinism	workers=1 versus workers=8 identical; Linux versus Windows CSV tolerance 1e-9 on a fixed fixture (run once and store the fixture)

WP-I4. Numerical diagnostics

Log, per rollout and method, the minimum eigenvalue of the covariance, the number of PSD floor events, the maximum condition number of S, and the count of eligible minutes. Write them to a CSV with the standard outputs. Add stress tests with extreme parameters (rho = 0.999999, sd = 1e-6, prior SD = 1e-6, V near singular) that must either run or fail with a clear FloatingPointError that is then counted in fit_failures.csv (never silently dropped).

WP-I5. Documentation

Docstrings for every public function stating inputs, outputs, causality and truth-access; docs/ARCHITECTURE.md with the data flow diagram (simulate to Rollout to adjust to detectors to calibration to evaluation); docs/GLOSSARY.md (canary, control, eligibility, release score, latched alarm, capped time, regime, oracle); docs/baselines.md (statistics with source equations); docs/CHANGELOG.md. Keep DESIGN_DECISIONS.md current.

WP-I6. On-the-fly data generation and storage budget

Each saved rollout costs about 0.7 MB (detector-visible arrays about 0.2 MB compressed plus ground-truth arrays a, spill, factor and noise of similar total size). The benchmark in E1 (about 45,000 rollouts) would need roughly 30 GB and the R1 grid about 65 GB. Because simulation is deterministic from (config, seed), large studies should **never store data**: they regenerate each rollout in memory from its seed, run the methods, and write only the CSV outputs.

1. Add `--generate-on-the-fly` to evaluate and validate-null: instead of reading data/<role>/*.npz they read the role manifest (config hash, seed purpose, counts, family weights), regenerate each rollout with simulate(), and discard it after scoring. audit_null already works this way.
2. Keep saved data only for the roles that define frozen artifacts (development, calibration, validation, test of the main benchmark) and for small fixtures. Store y as float32 and drop the noise and spill arrays from truth files unless a study needs them (they can be regenerated).
3. Test: on-the-fly and from-disk evaluation produce identical outputs for the same seeds (sorted CSV equality).
4. Record free disk space at the start of every long run and refuse to start a run whose estimated disk use exceeds 70% of free space (estimate = rollouts × 0.7 MB when storing).

Part 3. Validation and experiment programme

Experiments are grouped in tiers. Tier 0 repairs the evidence base (nothing here produces a headline number). Tier 1 is the fair benchmark and its ablations: these produce the paper's main claims. Tier 2 is robustness: these produce the failure map. Part 4 adds real-trace validation. Every experiment follows the same template: question, hypotheses stated before running, design, protocol with commands, outputs, analysis, decision rule, pitfalls, compute estimate. Hypotheses marked (H) are pre-registered predictions that may be wrong; being wrong is an acceptable result and is reported.

Compute assumption. A rollout costs roughly 0.35 to 1.0 s for all methods on one core of the user's CPU (sandbox timing: about 0.2 s for six methods; the real PC ran about 1.6× slower per update than the sandbox in the runtime benchmark). Estimates below use 0.8 s per rollout per full method set on one core and a 10-worker speed-up of about 8×. They are planning estimates, to be replaced by measured timings from WP-I2.

Tier 0. Repair the evidence base

X-0.1 Reproduction and determinism

Question. Does the first full run reproduce, and are results deterministic across worker counts and operating systems? **Protocol.** After WP-A3/A4/I1/I2: (1) rerun the plain pipeline with the legacy seeds at 100 pairs and compare per-rollout outputs with the first run's per_rollout_metrics.csv (same seeds must give identical scores; differences indicate a code change); (2) run with workers=1 and workers=8 and diff the CSVs; (3) run the 41-test suite on Linux (the sandbox) and Windows. **Decision rule.** Any non-identical output with identical seeds and code blocks all later work until explained. **Compute.** Minutes.

X-0.2 Explain the two anomalies and the hier = label-error invariance

(a) hier_arwhite equals hier_ssm exactly. Hypothesis (H): in a simulator with meas_sd = 0 the AR+white fit puts the white SD at its lower bound, so the two nuisance sets coincide to rounding. Protocol: print nuisance_main.json and nuisance_arwhite.json side by side (phi, q, v per class and the boundary-hit flags from diagnostics); run both detectors on 50 rollouts and assert whether score arrays are identical or differ at the 1e-12 level. If identical because of the boundary, say so and drop the ablation in the meas_sd=0 world; evaluate it in a world with meas_sd = 0.01 where it should differ.

(b) Per-regime calibration leaves step-regime false alerts at 6.8% to 9.0%. Protocol: (1) save the regime selector's confusion matrix (truth regime versus selected regime) over all 1000 evaluation nulls; (2) recompute thresholds with B = 2000 step nulls and re-validate on 2000 fresh step nulls; (3) apply the beta-binomial validity test (WP-C3) to each cell; (4) check that calibration and validation step nulls share the same generator settings (workload_step, beta_jitter, sd_host) and that the nuisance model did not see either; (5) compare the empirical CDFs of the calibration and validation maxima (KS). Interpretation table: selector errors present means a selector problem (WP-D4); validation passes with B = 2000 means the earlier excess was threshold sampling noise plus selector issues; still failing means a non-exchangeability source (look at host-slope draws, which are per-rollout random and can make the maxima distribution heavy-tailed: check the maxima's tail with a Hill plot).

(c) Label errors do not hurt the hierarchy. The first run shows hier_degraded (20% flipped labels in the detector) equal to hier (+0.10 min [−1.8, +2.0]), and S5 (30% label errors in metadata) success at 1.00. Hypothesis (H): at the selected hyperparameters (ρ=0.99, sd=0.001) the local-deviation coordinates barely move, so the class partition is nearly irrelevant and the hierarchy behaves like a pooled-plus-independent mixture. Protocol: run with sd_h_ratio in {0.25, 1, 4} and label error in {0, 0.3, 0.5}; examine posterior weight on class versus local coordinates. If the partition is irrelevant, the paper cannot claim the class structure is informative.

Antigravity prompt
Run X-0.2 (a), (b), (c) as three separate small studies using the existing code. Do not change any detector. Save all intermediate CSVs and a one-page generated summary per study (tables only, numbers from CSV). For (b) include the selector confusion matrix, the beta-binomial p-values, and the KS test. Report; stop.

X-0.3 Grid expansion on development data

Run fit on plain development data (500 pairs) with grids_v2 (WP-B1), K-fold selection (WP-B3), edge detection (WP-B2). Produce edge_selection.csv. If any primary method is at an edge, expand once or twice as specified, then freeze. Output the frozen model (kind = tuned), grid hash, candidate tables for every version, and a figure of the development objective versus each hyperparameter (profile plots) so interior optima are visible. **Compute.** With parallel and cache: about 20 to 60 minutes.

X-0.4 Plain-regime calibration validity (the gate that opens Tier 1)

Question. In the plain regime, does every method (including the new baselines, SSMs, ablations and combined detectors) have a valid false-alert rate? **Design.** Calibration: 2000 plain nulls (families mixed per plain_main.yaml). Validation: 2000 fresh plain nulls *per family* (7 families → 14,000 nulls). Methods: all registered methods. **Protocol.**

```
python -m canary_ssm.cli simulate --config configs/plain_main.yaml --role calibration --out data/plain/calibration
python -m canary_ssm.cli simulate --config configs/plain_main.yaml --role null_validation --out data/plain/null_validation  # per_
python -m canary_ssm.cli calibrate --data data/plain/calibration --model artifacts/model_v2 --out artifacts/policy_plain --regime
python -m canary_ssm.cli validate-null --data data/plain/null_validation --model artifacts/model_v2 --policy artifacts/policy_plai
```

Outputs. table_validation_by_family.csv (alerts, n, rate, CI, expected alerts, beta-binomial p), exchangeability_audit.csv (KS and Mann-Whitney per method), calibration_alpha_sweep.csv (nominal versus achieved at α in {0.01, 0.02, 0.05, 0.10}), threshold_uncertainty.csv, gate_status.json (G2 gate). **Decision rule.** Tier 1 starts only if every primary method passes G2 in every family and the number of failing cells is within the Bonferroni allowance. A failing method is fixed (investigate non-exchangeability) or dropped from the benchmark with the reason stated; it is never silently kept. **Compute.** 16,000 rollouts: about 3.5 hours single core, about 30 minutes with 10 workers.

Tier 1. The fair benchmark and ablations

E1 Main benchmark in the plain regime

Question. At equal validated false-alert rate in the plain regime, which methods detect weak persistent regressions fastest, and does the hierarchy help?

Hypotheses (pre-registered, directions). (H1) The best SSM variant has lower mean capped time than the best tuned classical baseline in S1, S2, S3, S5; the difference exceeds the 3-minute margin. (H2) Hierarchical beats independent in S2 and S5 and loses or ties in S1, S4 and single-canary; pooled beats both in S3. (H3) Structure-adaptive detectors (D8, D9) recover the best of the three structures in every family within the margin. (H4) Strong baselines (ar_cusum, glr_window, sparse_mix_glr, msprt) narrow but do not close the gap to oracle_bayes in S2. All four may fail; failure is reported with the same prominence.

Design. Methods: matched_contrast, adj_ewma, adj_cusum, topk_cusum, wmean, ar_cusum, glr_window, sparse_mix_glr, bocpd, msprt, mw_judge, pooled_ssm, indep_ssm, hier_ssm, rs_hier_ssm (if D3 adopted), enum_ssm (D8), bma_ssm (D9), combo_* detectors, oracle_* references (labelled). All methods tuned on the same development set with equal-effort protocol, calibrated on the same plain calibration nulls, evaluated on the same test pairs.

Block	Factor levels	Pairs per cell	Cells
Core scenarios (noise 0.02, effect 3%, duration 45)	S1 sparse (3 of 10), S2 class-wide (5 of 10), S3 broad (10 of 10), S4 weak sparse (2.5%), S5 class-wide with 30% label errors, single canary (4%), transient (10 min, challenge set)	1000	7
Noise sweep (S1, S2, S3)	innovation SD 0.01, 0.02, 0.04	600	9
Effect sweep (S1, S2)	effect 2.5%, 3%, 5%, 10%	600	8
Affected-count sweep	1, 2, 3, 5, 10 canaries (random subset)	600	5
Class split sweep	two classes 5/5, 8/2, 9/1; three classes 4/3/3	600	4

Total harmed pairs ≈ 7×1000 + 26×600 = 22,600 pairs, or about 45,000 rollouts. Planning estimate: 45,000 × 0.8 s = 10 hours single core, about 1.3 hours on 10 workers (add 50% for the heavier D3/D8/D9 methods). Test seeds from the registry; test data generated only after prereg-v1 is tagged.

Analysis. Per-cell mean capped time, miss, premature and late rates, success at 15/30/60 minutes, cumulative incidence curves (F3); paired contrasts per prereg with Holm adjustment (F2); equivalence statements;

efficiency ratio versus oracles (E2); scenario-balanced pooled contrasts; between-scenario heterogeneity. **Decision rule.** Map results to the decision tree in Section 1.6 and write the branch into docs/DECISION.md before any manuscript text is drafted.

Pitfalls. (i) Comparing methods that did not pass X-0.4. (ii) Reporting conditional delay. (iii) Pooling scenario cells with unequal n without weights. (iv) Choosing the 'best baseline' on the test set: it must be the baseline with the best *development* objective. (v) Letting an oracle appear in a ranking. (vi) Forgetting that premature alarms are failures.

Antigravity prompt
Run E1 only after X-0.4 passes and prereg-v1 is tagged. Generate test data from configs/plain_main.yaml and the sweep configs listed in docs. Evaluate with the frozen tuned model and policy. Compute every pre-registered contrast with Holm adjustment. Produce the generated report and figures. Do not interpret; write results/e1/RESULTS_TABLES_ONLY.md and stop.

E2 Ablation suite

Question. Which model components carry the performance, measured with ablations that are not structurally handicapped? Each ablation retunes what it needs on development data and is recalibrated.

ID	Ablation	Levels	Primary contrast
A1	Persistence (fair, B4)	rho in {0, 0.5, 0.8, 0.9, 0.95, 0.99, 0.999}; plus windowed-mean comparator	capped time versus rho; hier at optimum versus rho = 0
A2	Latent factor adjustment	factor rank 0 versus 1 (rank 2 if G-extension with two factors)	hier versus rank0 across shock sizes {0, 0.05, 0.15}
A3	Noise model	ar_only, ar_white, white; evaluated at meas_sd in {0, 0.01}	hier versus each; does the white component ever help?
A4	Class labels	label error {0, 10, 30, 50%}; sd_h_ratio {0.25, 0.5, 1, 2, 4}; G in {1, 2, 3, 5}	is the partition informative? (relates to X-0.2c)
A5	Slope handling	pre, pooled_dev, eb_controls (D1), rs_ssm (D3)	per-rollout residual SD and plain-null power cost
A6	Robust likelihood	Gaussian, Huber c in {2, 3, 4}, Student-t nu in {3, 6, 10}	plain power cost versus heavy-tail benefit (R2)
A7	Eligibility rule	coverage in {0.5, 0.8, 0.95}; min controls in {1, 3, 5}	abstention versus power
A8	Prior and effect-scale	prior SD {0.005, 0.01, 0.02, 0.03, 0.05}; effect-size mismatch (true 1.5×, 2×)	sensitivity to the prior

Design. Core scenarios S1, S2, S3, S5, noise 0.02, 600 pairs per cell. **Analysis.** Paired contrast of each ablated variant against the full model; interpret a component as 'necessary' only if the paired interval excludes the ±3-minute margin and the ablated variant has been given a fair tuning budget.

E3 Hyperparameter sensitivity and tuning-budget parity

(a) Response surfaces. For hier_ssm: heatmaps of development and test capped time over (rho, sd_g) at the selected prior SD, and over (prior SD, sd_g) at the selected rho. Question: is the optimum a knife edge? **(b) Budget parity.** Random search with M in {6, 12, 24, 48, 96} candidates per family, 20 repetitions; plot test capped time versus M for every family; test whether the family ranking changes with M. **(c) Constraint sensitivity.** Constrained versus unconstrained long-run-SD grid (WP-B1). **(d) Misspecified-tuning robustness.** Tune on noise 0.02 and test at 0.01 and 0.04 (transfer of tuned hyperparameters). **Decision rule.** If the ranking depends on M or the constraint, the paper reports it as a limitation and states the regime in which the ranking holds.

E4 Power surfaces and minimum detectable effect

Question. What is the smallest persistent regression each method detects within 30 and 60 minutes at 80% power and α = 0.05, as a function of noise, number of affected canaries and class structure? **Design.** effect in {1.5% (non-harmful), 2%, 2.5%, 3%, 4%, 5%, 7.5%, 10%} × noise {0.01, 0.02, 0.04} × affected {1, 3, 5, 10, class-wide} × 300 pairs per cell (120 cells × 300 = 36,000 pairs; reduce by dropping the 1.5% level for broad cases or by using adaptive sampling that stops a cell when its CI is narrow). **Analysis.** Fit a probit model P(success by 30 min) = Φ(a + b log(effect) + ...) per method and scenario with cluster-robust SEs; report the

effect for 80% power with a bootstrap interval. Include effects at or below κ: success there is defined by the event rule (no qualifying event for effect ≤ κ), so the relevant output is the false-alert rate on sub-κ effects (alarming on non-harmful change is a false alert in the event definition; report it separately as 'alarms on sub-boundary effects').

E5 Power versus false-alert curves

Question. Are conclusions about ranking stable across operating points? **Design.** Re-use E1 test outputs; thresholds at α in {0.01, 0.02, 0.05, 0.10, 0.20} from the stored calibration maxima (WP-C2). **Outputs.** power-at-30-minutes versus α curves with bootstrap bands; area under the curve over α in [0, 0.2]; the α at which each pair of methods crosses (if any). **Decision rule.** A ranking that reverses between α = 0.01 and α = 0.10 is reported as operating-point dependent; the abstract may not claim a general ordering. **Pitfall.** α = 0.01 needs at least 2000 calibration nulls and about 5000 validation nulls to resolve; mark cells below that as exploratory.

E6 Host localization, FDR and probability calibration

Question. Can the detectors say which hosts are affected, and are the probabilities trustworthy? **Design.** Harmed rollouts of E1; landmarks at onset+15, +30, +60. **Analysis (WP-F5).** top-3 recall, F1 at the dev-selected cutoff, host AUC, precision-recall curve; Bayesian-FDR selection and achieved false discovery proportion; reliability diagrams with Wilson intervals; Brier decomposition. **Hypotheses.** (H) The hierarchical model is best calibrated in class-wide harm; independent hosts are best at localizing sparse harm; pooled cannot localize. **Pitfall.** Host-level probabilities are model-conditional; if reliability is poor under misspecification (R2) say so.

E7 Decision-theoretic evaluation

Question. For realistic cost ratios, which method minimises expected loss? **Design.** Cost ratio r = C_fa / C_miss over {0.1, 0.3, 1, 3, 10, 30, 100, 300, 1000}; per method, choose α on development data to minimise development loss, then evaluate on test. Harm-minutes use equal traffic share per canary by default and a variant weighting by exposure fraction in staged rollouts (G7). **Outputs.** loss versus r curves with bands; the range of r in which each method is best; net-benefit decision curves. **Decision rule.** State the cost regime where the hierarchy wins and where it does not; do not pick one ratio.

E8 Advisory policy and flicker

Question. What is the operational cost of a reversible pause/promote policy? **Design.** The advisory policy of WP-F7 with hyst_frac and n_clear selected on development data; null rollouts for reversals and pause minutes; harmed rollouts for time to first pause and time paused after recovery. **Output.** flicker table by method; trade-off plot of reversals versus time-to-first-pause. **Decision rule.** A method whose first-pause rate is valid (same threshold as the latched policy) but flickers more than baselines at equal power is reported as such.

Tier 2. Robustness studies (the failure map)

Robustness studies never tune. They take the frozen tuned model from E1 and its plain policy, then expose it to environments that differ from development. Each reports, for every method, four numbers per environment: false-alert rate with nominal thresholds (transfer), false-alert rate with thresholds recalibrated on matched nulls (WP-C5), power with the matched thresholds, and abstention fraction. A method is called 'robust' to a perturbation only if its transfer false-alert rate passes the G2 gate across the perturbation range. 'Robust' never refers to the absence of alerts.

R1 Shared workload steps, ramps and slope heterogeneity (the main failure)

Question. How do false alerts and power change as a concurrent workload change grows, and which adjustment (pre-period slope, pooled slope, EB shrinkage, RS-SSM) and policy (ignore, abstain, regime threshold) limit the damage?

Hypotheses. (H1) False-alert rate of plug-in methods increases monotonically in |step| and in slope heterogeneity. (H2) EB shrinkage reduces the per-rollout residual SD versus per-host pre-period slopes by at least 20% under heterogeneity. (H3) RS-SSM reduces step false alerts relative to hier_ssm at a power cost under 30% (relative increase in capped time) in the plain regime. (H4) The support-based policies P1/P2 keep

false alerts valid in the step regime at the price of abstention minutes that grow with |step|; their power in the step regime is lower than in plain. (H5) Longer pre-periods (T_pre) shrink slope errors as 1/sqrt(T_pre) and eliminate most of the problem at T_pre = 1440. Any of these may fail.

Design. Factors: step size {0, 0.1, 0.2, 0.3365, 0.5, 0.7} (log units; 0.3365 ≈ +40% workload); shape {step, ramp over 5, 15, 30, 60 minutes}; onset offset relative to t0 {−30, 0, +30, +60} (offset −30 means the shift occurs in the pre-period, so the pre-period fit sees it); slope heterogeneity beta_jitter {0, 0.03, 0.08, 0.15}; post-shift workload variability (AR innovation SD multiplier {1, 3}); pre-period length {180, 360, 720, 1440} (requires WP-G6); adjustment {pre, pooled_dev, eb_controls}; detector {matched_contrast, adj_cusum, topk_cusum, ar_cusum, glr_window, indep_ssm, hier_ssm, rs_hier_ssm, enum_ssm, bma_ssm}; policy {P0, P1, P2}. A full factorial is far too large; use these designs: (a) a main-effects sweep, one factor at a time around the centre (step 0.3365, step shape, offset 0, jitter 0.03, T_pre 180, adjustment pre, policy P0); (b) the full {step size × jitter} grid for the three adjustments; (c) the {pre-period length × jitter} grid for the step 0.3365; (d) the policy comparison at step sizes {0.1, 0.3365, 0.7}. Null rollouts per cell: 1000 for false-alert cells (validity needs n), harmed rollouts per cell: 300 for power cells.

Outputs. false-alert heatmaps (step × jitter) per detector and adjustment with G2 pass/fail overlay; per-rollout post-minus-pre residual shift SD and RMS (the Experiment 8 diagnostic, retained), β error spread; power cost curves (capped time in plain versus step regime); regime-flag ROC (D4); abstention minutes; runtime of RS-SSM. **Analysis.** For each cell a validity test (WP-C3) comparing the stress false-alert rate against a policy calibrated on plain nulls; response curves false-alert rate versus step size with Wilson bands; a regression of false-alert logit on (step, jitter, step × jitter, 1/sqrt(T_pre)) to test the mechanism: the interaction step × jitter should be the dominant term if slope heterogeneity drives the failure. **Decision rule.** If no variant passes the gate at step 0.3365 with jitter 0.08, the paper states that concurrent large workload changes cannot be handled at the 5% budget with these methods in this simulator and recommends abstention (P1) with the measured abstention cost.

```
# configs/studies/R1.yaml (example)
study: R1
base: {sd_host: 0.02, n_affected: 3, effect_pct: 3.0, duration: 45}
factors:
  workload_step: [0.0, 0.1, 0.2, 0.3365, 0.5, 0.7]
  beta_jitter:   [0.0, 0.03, 0.08, 0.15]
  step_onset_offset: [0]          # new field (G2/G6): offset of the step relative to t0
  step_ramp_len: [0]
adjustments: [pre, pooled_dev, eb_controls]
detectors: [matched_contrast, adj_cusum, topk_cusum, indep_ssm, hier_ssm, rs_hier_ssm]
policies: [P0, P1, P2]
null_per_cell: 1000
harm_per_cell: 300
seed_purpose: study_R1
```

Compute. Grid (b): 6 × 4 × 3 adjustments = 72 cells × 1300 rollouts ≈ 94,000 rollouts per detector set; the detector set shares residuals so cost is about 0.8 s per rollout: ≈ 21 hours single core, about 2.7 hours on 10 workers. Grid (c) adds about 40% more. Use cached residuals.

Antigravity prompt
Implement the new simulator fields needed by R1 (step onset offset, ramp length, post-step workload variability) as WP-G2 fields with defaults that reproduce legacy output exactly and a bit-identity test. Then run R1 designs (a) and (b) only. Save every table as CSV; generate the heatmaps and the regression table from CSV. Do not interpret. Report runtime per design.

R2 Calibration transfer: noise persistence, heavy tails, loading shifts, nonlinearity, white noise

Question. Which environmental perturbations break calibration, and which of {matched recalibration, controls-based noise estimation, robust likelihood, parameter-uncertainty SSM} repairs which? **Perturbations.** φe in {0.3, 0.6, 0.8, 0.9}; Student-t degrees of freedom in {3, 5, 10, ∞}; loading shift in {0.8, 1.0, 1.3, 1.6}; shock size in {0, 0.05, 0.15, 0.3}; loading dispersion lam_host_sd in {0.02, 0.05, 0.15, 0.3}; nonlinear workload gamma in {0, 0.1, 0.3} (with workload step 0.3365 so the response is exercised); white measurement noise in {0, 0.01, 0.02}. One factor at a time, plus three combined worst cases.

Arms. (A) nominal thresholds, nominal nuisance; (B) matched-environment recalibration (thresholds from 1000 nulls of the perturbed environment; nuisance unchanged); (C) matched recalibration with refit nuisance (WP-C5); (D) nuisance from controls (WP-D2: controls_pre and controls_online) with nominal thresholds; (E) robust likelihood (D5) with nominal thresholds and with matched thresholds; (F) RS-SSM (D3) with nominal thresholds. **Hypotheses.** (H1) Arm B restores validity for every method for every perturbation except nonlinear workload (a mean misspecification that matched thresholds can absorb only if the perturbed nulls match the deployment exactly). (H2) Arm D makes the SSM statistic nearly pivotal in φe. (H3) Student-t tails inflate matched contrast and the SSMs, and Arm E with matched thresholds restores validity but costs power. (H4) Loading shift and shock size interact: big shocks with dispersed loadings inflate false alerts; Arm F reduces it.

Design. 1000 nulls per (perturbation level × arm × method) for false-alert cells; 300 harmed per cell for power; calibration nulls for arms B, C, E: 1000 per environment. **Analysis.** Validity test per cell; 'transfer curves' of false-alert rate versus perturbation magnitude with the valid band shaded; table of 'minimal repair': for each perturbation, the cheapest arm that passes. **Compute.** roughly 7 perturbation families × 4 levels × 6 arms × 1300 rollouts ≈ 218,000 rollouts: about 48 hours single core, about 6 hours on 10 workers. Prioritise: φe, heavy tails and loading/shock first (the three that failed in the first run).

R3 Control contamination and interference

Question. How badly does the factor and the matched contrast degrade when some controls are affected by the canary (spillover through shared caches, load balancers, dependencies)? **Design.** contam_frac in {0, 0.1, 0.2, 0.4}, contam_scale (fraction of the canary effect reaching contaminated controls) in {0.25, 0.5, 1.0}; also the reverse (canary traffic affecting controls through shared downstream). **Mechanism to measure.** The factor estimate absorbs the shared part of the effect (rank-one projection), so the canary residual shrinks: sensitivity is expected to be monotone in contam_frac × contam_scale. **Mitigations to evaluate.** (i) robust factor weights (down-weight controls whose residual correlates with canary exposure after the rollout: diagnostic only; must be causal and truth-free); (ii) exclusion of known contaminated controls (given as metadata); (iii) trimmed factor projection (drop controls with the largest post-rollout residual trend using data to time t). **Report.** detection power loss, false-alert rates (contamination should reduce power rather than inflate false alerts, but verify), and a diagnostic statistic that flags contamination (e.g. rise of mean control residual after rollout). **Claim discipline.** State that attribution assumes uncontaminated controls (Experiment-6 handoff conclusion) and show the cost when it fails.

R4 Missingness and abstention

Design. MCAR rates {0, 5, 20, 40%}; class-wide canary dropout bursts of {5, 10, 20} minutes; late arrival fraction {0, 10, 30%} with delay {3, 10} minutes; informative missingness (dropout probability increases with the effect or with load; G8). **Report.** abstention fraction (eligible minutes), time to first eligible minute after t0, power among and including abstentions, false-alert rate. **Hypothesis.** At 20% MCAR with 10 canaries the 80% canary-coverage rule makes the detector abstain about a third of the minutes (first-run sandbox observation); a coverage rule of 50% restores most minutes at a measurable calibration cost. **Decision rule.** Select the eligibility rule from E2/A7 on development data; report the trade-off curve in the paper. Do not call a detector robust to missingness without its power curve.

R5 Shared incidents and negative controls

Question. Do the detectors stay quiet when a platform incident affects canaries and controls alike? **Design (G3).** Incident shapes {pulse 5, 15 minutes; ramp 30; plateau 90} with sizes {2%, 5%, 10%} and loading dispersion {0.05, 0.15}; no local harm (nulls) and local harm plus incident (mixtures); incident starting before, at and after t0. **Report.** false-alert rate in no-harm incidents (should be at or below α if the factor absorbs the incident), power in mixtures, and the dependence on the factor estimation precision U_t. **Hypothesis (H).** The rank-one factor handles homogeneous incidents but not incidents with host-specific loading patterns that differ between canaries and controls; two-factor extensions or RS-SSM (loading error states) are the mitigations to evaluate.

R6 Effect shape misspecification: ramps, flapping, load-dependent harm, improvements

Design (G4). Shapes {abrupt, ramp over 10/30/60 minutes, flap with period 10/20/40 minutes and duty cycle 50%, load-dependent (effect proportional to workload level with coefficient chosen to give 3% at the mean load), negative effect (−3%), sub-boundary effect (1.5%)}. **Hypotheses.** (H1) High persistence hurts on flapping effects relative to windowed means; (H2) high persistence is robust to slow ramps but adds latency; (H3) improvements do not trigger alarms (directional score); (H4) sub-boundary effects trigger alarms at a rate that should be reported (they are false alerts under the event definition even though a real change occurred). **Report.** capped time, miss, premature, late rates per shape; alarms on improvements and sub-boundary effects.

R7 Scalability and numerical behaviour

Design. n_canary in {10, 50, 100, 200}, hosts up to 1000, metrics d in {1, 2, 4} (requires the multi-metric extension, R9). Measure per-update runtime distribution (median, p95, p99, max) over 5000 updates per setting, memory, covariance minimum eigenvalue, PSD floor events, and fit failures for hier_ssm, indep_ssm, rs_hier_ssm, enum_ssm, bma_ssm. Record CPU model, library versions (BLAS vendor), threading settings. **Pass condition.** p99 update below 1 s at n = 10 (the plan's gate), and a stated scaling exponent from a log-log fit. First-run reference: p99 0.40 ms at n = 10 and 54 ms at n = 100 for hier_ssm.

R8 Class structure and metadata quality

Design. Number of classes G in {1, 2, 3, 5, 10}; unbalanced splits; singleton classes; class label error {0, 10, 30, 50%} (also random labels); classes that are real (different μ, σ, φ) versus arbitrary; **data-driven classes**: cluster canaries by pre-period residual correlation or by workload slope (k-means or hierarchical clustering with G chosen by silhouette) and compare to metadata classes. **Question.** When is the hierarchy's partition informative, and is metadata required? **Hypothesis (H).** With real classes (distinct effects by class, G = 2) the hierarchy beats independent hosts in class-wide harm; with arbitrary classes it does not, and with 50% label error it matches independent hosts. Ties with X-0.2(c).

R9 Multi-metric extension

Motivation. Real canary analysis monitors several correlated metrics (latency, error rate, CPU, saturation). **Implementation (new WP, optional for the first paper).** Extend the simulator to d correlated metrics with metric-specific loadings and noise; vector observations in the Kalman filter (the code already supports H with several rows per host); effect per metric with shared or independent structure; multiplicity across metrics handled by calibrating the release score of the *combined* max over metrics. **Design.** d in {1, 2, 4}; correlation in {0, 0.3, 0.7}; effect in one metric or all. **Report.** power, calibration, runtime. Mark this a stretch goal: the single-metric paper is complete without it, but a reviewer will ask.

Part 4. Real-trace validation

Every result so far comes from a simulator whose assumptions the models share. A reviewer will say, correctly, that this proves little about production. Real traces cannot supply labelled canary regressions (I am not aware of a public labelled canary-analysis dataset; Antigravity must search, report what it finds, and the paper must say so if none exists). They can supply two things that matter: **real null behaviour** (noise, autocorrelation, tails, incidents, workload, host heterogeneity) and a substrate for **semi-synthetic injection** of known effects. This part specifies that bridge.

Note. I cannot verify dataset availability, schemas, licences or download locations from here. The names below are starting points. Antigravity must open each source, record the exact schema and licence in docs/data_sources.md, and stop and report if a dataset is unsuitable. Do not guess field names.

4.1 Candidate sources to inspect

Source (to verify)	Why it might help	Likely limitations to check
Google cluster-usage traces (2011 and 2019 releases)	Many machines, long periods, per-task resource usage; some versions record a performance indicator (cycles per instruction) and usage series per machine or task	Native sample window (for example five minutes) coarser than one minute; no latency; no canary labels; incidents unlabelled
Alibaba cluster traces	Machine-level CPU, memory and network series for many hosts; some releases include online-service containers	Same coarse resolution and no labels; check licence and which fields exist
Azure public dataset releases (VM CPU readings and related)	Large VM populations with utilisation series	Utilisation not latency; need a latency-like proxy
Anomaly-detection benchmark series (for example Numenta Anomaly Benchmark, Yahoo Webscope S5)	Real non-Gaussian series with labelled anomalies, good for tail behaviour	Single series, not fleets with controls: only useful for noise realism, not for canary structure
The user's own services or logs, if any	True canary structure and real rollouts	Access and privacy; may be impossible; check before planning on it

4.2 From a trace to the Rollout schema

1. **Select a fleet.** Choose hosts that serve the same role over the same period with at least 95% sample coverage. Record the selection rule and every excluded host with a reason.
2. **Define y and x.** y_it must be a latency-like quantity (use a performance proxy such as cycles per instruction if present, else a utilisation or response-related field) transformed with the logarithm after confirming positivity. x_t must be a workload covariate (aggregate request, task or arrival counts, or fleet-total utilisation), log-transformed. Write down why each is a sensible proxy and what it is not.
3. **Time grid.** If the native resolution is coarser than one minute, do not interpolate to one minute. Define the grid in native steps and rescale the detector time constants (pre-period 36 steps, horizon 24 steps for a 5-minute grid, which keeps the 3-hour and 2-hour spans) and the effect duration. Document that results at a different resolution are not directly comparable to the simulator's.
4. **Classes.** Assign classes by machine type or platform from the trace's metadata; if none, cluster hosts on pre-period behaviour (R8) and report the clustering.
5. **Write the loader** canary_ssm/semisynth.py: `make_rollouts(trace, n_rollouts, canary_fraction, class_rule, t_pre, horizon, seed) -> list[Rollout]`, producing Rollout objects with no truth fields; truth (the injected effect and the canary set) goes into a separate Truth-compatible object in a sibling directory exactly as in the simulator.

4.3 Placebo calibration check on real nulls (RT-1)

Question. What false-alert rate do the methods have on real healthy data when calibrated on simulated nulls, and on real nulls from an earlier period?

1. Draw placebo rollouts: random start times in the trace and random canary subsets (about 10% of hosts, stratified by class). No injection. Real incidents inside the window are part of the null and are counted as false alerts: that is what an operator would experience. Record which placebo windows overlap known incident periods if the trace labels any.
2. Temporal split: calibration windows from the first 50% of the trace, validation windows from the last 50%, with a gap larger than the window length between them. Never split randomly across time.
3. Calibrate three ways: (a) simulator-calibrated thresholds from E1; (b) thresholds from real calibration placebos; (c) real placebo thresholds with nuisance refitted on the real calibration period. Validate each on real validation placebos with the beta-binomial test (WP-C3).
4. Dependence: placebo windows that overlap in time share data. Use non-overlapping windows or a block bootstrap by calendar day for intervals; report the effective sample size.

Outputs. real_null_rates.csv with intervals; transfer table (a) versus (b) versus (c); diagnostic plots of the real residual ACF, QQ plot, tail index, loading dispersion, slope heterogeneity, and workload variance compared with the simulator's. **Decision rule.** If (a) fails and (b) passes, the paper's message is the recalibration protocol; if (b) also fails, the statistic is not exchangeable on real data and the paper reports where it breaks.

4.4 Semi-synthetic injection (RT-2)

Take the same placebo windows and, in a paired copy, add the log of (1 + δ) to the chosen canaries' y from the effect onset for D minutes (shapes from G4). The pair shares every exogenous array; the harmed copy differs only by the injection (the simulator's invariant). Effects δ in {2%, 3%, 5%, 10%}, affected counts as in E1, durations 45 and 90 minutes. **Report** power, capped time and paired contrasts exactly as in E1, with real noise. **Questions.** Does the ranking of E1 survive real noise? Which assumption of the simulator mattered most? Use the 'calibrated simulation' approach to answer the second: fit simulator parameters (φ, tails, loading dispersion, slope heterogeneity, workload variance) to the real null statistics, rerun E1 under the fitted simulator, and compare with RT-2. Agreement means the simulator is adequate for its purpose; disagreement identifies the assumption to improve.

4.5 Simulator fidelity table (RT-3)

Statistic	Simulator default	Real trace (to measure)	Comment
Lag-1 autocorrelation of host residuals (after workload and factor adjustment)	0.6	?	Drives CUSUM and AR handling
Excess kurtosis of host residuals	0 (Gaussian)	?	Heavy tails break calibration (R2)
Between-host SD of workload slopes	0.03 (default jitter)	?	Heterogeneity breaks pooled slopes (R1)
Between-host SD of factor loadings	0.05	?	Loading error times shock
Pre-period workload variance	0.0095	?	Slope identification
Fraction of windows with shared incidents	shock at t0 only	?	R5
Missingness and late data	none by default	?	R4

4.6 Rules specific to real data

- No tuning on real validation data. Hyperparameters come from simulator development data or from the real calibration half, never from the real validation half.
- Record licences and any terms of use; do not redistribute raw data; the reproduction package contains the loader and a download script, not the data.
- No personal data. If internal data is ever used, obtain permission, anonymise host names, and keep the raw data out of the repository.
- State clearly in the paper that the effects are injected, so the study demonstrates calibration under real nulls and power under known injections, not detection of real regressions.

Part 5. Paper kit

5.1 What can be a paper, and how to decide

Use the decision tree in Section 1.6 after experiment E1. Whatever branch applies, the paper's contributions should be framed as **(1) a rigorous evaluation protocol** for canary analysis under shared workload change (paired simulator with isolated truth, exact calibration validity test, equal-false-alert comparison, capped-time metric), **(2) a failure map** (when calibration and power break and why), and **(3)** the method results in whichever direction they come out. This framing is robust to a negative outcome, because contributions 1 and 2 stand either way.

5.2 Title and abstract templates (fill only from generated numbers)

Working titles (pick after E1). (A) 'Calibrated sequential detection of weak canary regressions under shared workload change: a state-space evaluation'. (B) 'When does hierarchy help? Structure-adaptive state-space canary analysis'. (C) 'Calibration, not model complexity: a benchmark of canary regression detectors'. (D) 'Detecting weak persistent regressions in canary releases: an equal-false-alert comparison of sequential methods'.

```
Abstract template (each [ ] is filled by a macro from a CSV; no hand-typed numbers)
Problem: Canary analysis must detect weak, persistent latency regressions on an unknown subset of canary hosts while workload shif
Gap: [one sentence, from related work check].
Method/protocol: We evaluate [N] detectors, including [state-space structures] and [classical baselines], in a paired simulator wi
  false-alert budget validated by an exact finite-sample test, using horizon-capped time to detection as the primary metric.
Results: In the plain regime, [best method] reduced capped time by [x] minutes [CI] versus [best baseline] (class-wide harm) ... ;
Failure map: Under a concurrent workload step, false-alert rates rose to [range] for [methods]; [mitigation] restored validity at
Real-trace check: [placebo false-alert rates; injected-effect power].
Conclusion: [conditional statement]. Limitations: simulation and injected effects only; no labelled real canary data.
```

5.3 Section outline with source of each claim

Section	Content	Evidence (experiment / WP)
1 Introduction	Canary analysis problem; why shared workload change and weak effects are hard; contributions	Part 1; Figure 1 (concept)
2 Related work	Sequential change detection (CUSUM, EWMA, GLR, mixture GLR, BOCPD), always-valid inference, canary analysis systems, dynamic factor and structural time-series models, causal-impact methods, sparse multi-stream detection	Verified citation list (5.7)
3 Problem and metrics	Setting, harm boundary, event definition, outcome classes, capped time, threshold rule	Section 0.3; F1-F3
4 Methods	Adjustment pipeline, SSM structures (pooled, independent, hierarchical), enumerated-hypothesis and averaged variants, parameter-uncertainty filter, regime policies, baselines and oracles	WP-D, WP-E
5 Evaluation protocol	Simulator, roles, seeds, calibration, exact validity test, tuning protocol and budget parity, pre-registration, statistics	WP-A to C, F; X-0.4
6 Results: plain regime	E1 main benchmark, E2 ablations, E3 sensitivity, E4 power surfaces, E5 power-α, E6 localization, E7 decision loss	Tier 1
7 Results: failure map	R1 step and heterogeneity, R2 transfer and recalibration, R3-R9	Tier 2
8 Real-trace validation	RT-1 placebo, RT-2 injection, RT-3 fidelity	Part 4
9 Discussion	What the hierarchy buys; what persistence means; why calibration is the crux; operational guidance (recalibrate per environment, abstain under shifts)	Interpretation rules (5.8)
10 Threats to validity and limitations	Section 5.9	
11 Reproducibility	Repository, seeds, manifests, one-command reproduction	5.10

5.4 Figures (each generated from CSV by report.py; vector PDF; CSV twin saved)

Fig	Content	Source
F1	Concept: shared workload step hides a local regression (schematic; numbers illustrative)	Hand-drawn; labelled illustrative
F2	Data flow: simulate to Rollout to adjust to detectors to calibration to evaluation, with truth isolation	docs/ARCHITECTURE.md
F3	Calibration validity: achieved false-alert rate per method and family with the beta-binomial acceptance band (forest plot)	X-0.4
F4	Cumulative incidence of detection versus minutes since onset, per method, per core scenario, with bands	E1, F3
F5	Mean capped time per scenario and method with intervals (bar or dot plot), oracle references hatched	E1
F6	Paired-contrast forest plot with margin band and Holm-adjusted significance	E1, F1/F2
F7	Ablation panel: persistence curve over rho; factor, noise, labels, slope handling, robust likelihood	E2
F8	Power surfaces and minimum detectable effect versus noise and number affected	E4
F9	Power versus false-alert curves and AUC	E5
F10	Host localization PR curves and probability reliability diagrams with Wilson intervals	E6
F11	Expected loss versus cost ratio; decision curves	E7
F12	False-alert heatmaps over step size and slope heterogeneity, per adjustment and detector, with pass/fail overlay	R1
F13	Transfer curves: false-alert rate versus perturbation (φe, tails, loadings, shock) for nominal versus recalibrated thresholds	R2
F14	Real-trace placebo false-alert rates and injected-effect power versus simulator	RT-1, RT-2
F15	Runtime scaling (log-log) with p99 bands	R7

5.5 Tables

- T1 Notation and frozen definitions. T2 Methods list with hyperparameter grids and selected values (manifest). T3 Scenario design (core, sweeps). T4 Calibration validity table with beta-binomial p-values. T5 Main results per scenario (capped time, miss, premature, late, success at 15/30/60). T6 Pre-registered contrasts with Holm-adjusted p and equivalence verdicts. T7 Ablation summary. T8 Failure map: for each perturbation, which methods fail and the minimal repair. T9 Real-trace results. T10 Runtime. T11 Threats and mitigations. Appendix tables: per-family nulls, abstention, stratified results, hyperparameter edge reports, fit failures.

5.6 Theory notes to write (each verified numerically before it is claimed)

1. **T1 Exact validity of order-statistic calibration.** State and prove: under exchangeability and continuity, the threshold is the k-th smallest of B maxima; the exceedance probability of a new maximum is Beta(B+1−k, k); the validation alert count is beta-binomial (WP-C3 derivation). Corollaries: the policy's true false-alert rate has standard deviation about sqrt(α(1−α)/B); the minimum B is (1−α)/α; ties make the test conservative. Verify by simulation in a test. This is the paper's cleanest rigorous contribution.
2. **T2 Relation of CUSUM and the persistent-effect filter.** A scalar Gaussian level-shift filter with effect prior N(0, τ^2), ρ = 1 and no process noise has posterior mean $S_n / (n + \sigma^2/\tau^2)$ and is a function of the running sum S_n of residuals; CUSUM maximises over change times of partial sums with drift subtraction; mSPRT uses S_n^2 through a mixture likelihood. State the relation and where they differ (unknown change time, drift subtraction, directional prior), verified numerically on simulated data. Do not claim optimality.
3. **T3 Identifiability of effect versus coefficient error.** Observations of an effect level a, an intercept error δμ and a slope error δβ times a regressor u_t are collinear when u_t is constant after t0: any (a, δμ, δβ)

with a + δμ + δβ u = const fits equally. The filter separates them only through priors; when u_t varies after the shift, δβ is identified from Cov(residual, u). Quantify by the posterior correlation in RS-SSM and by R1's post-shift variability factor.

4. **T4 Delay approximations.** For a standardised persistent shift δ, a CUSUM with reference value k and threshold h has expected delay approximately h / (δ − k) for large h (verify the form against the sequential-analysis literature before citing; use it only to explain simulated trends). Compare with simulated delays across the effect sweep.
5. **T5 Effective sample size under AR noise.** For AR(1) noise with coefficient φ the variance of a mean of n observations is inflated by about (1+φ)/(1−φ); this explains why unadjusted EWMA/CUSUM thresholds are large and why prewhitened CUSUM (ar_cusum) changes the effective drift to (1−φ)δ. Verify numerically.

5.7 Related work to read and verify (do not cite from memory)

The research report already verified: Adams and MacKay (2007, BOCPD, arXiv:0710.3742); Xie and Siegmund (2013, sequential multi-sensor change-point detection, Annals of Statistics 41(2), 670-692); Eichler, Motta and von Sachs (2011, dynamic factor models for non-stationary series, Journal of Econometrics 163(1), 51-70; note the first report mis-attributed the authors); Barigozzi, Cho and Fryzlewicz (2018, simultaneous multiple change-point and factor analysis, Journal of Econometrics 206(1), 187-225); Brodersen et al. (2015, causal impact with Bayesian structural time series, Annals of Applied Statistics 9(1), 247-274); Candes, Li, Ma and Wright (2011, robust PCA, Journal of the ACM 58(3)); the Google SRE Workbook chapter on canarying releases; and Google's Canary Analysis Service description. The additional sources below are ones I believe exist and are relevant; Antigravity must locate each, record the DOI or URL, and read it before it is cited:

- Classical sequential change detection: Page (1954, CUSUM, Biometrika); Roberts (1959, EWMA, Technometrics); Lorden (1971) and Moustakides (1986) on optimality; Siegmund's sequential analysis text; Alwan and Roberts on residual charts for autocorrelated data.
- Always-valid sequential inference: Johari, Pekelis and Walsh (always valid inference; 'Peeking at A/B tests', KDD 2017).
- State-space methods: Kalman (1960); West and Harrison, Bayesian Forecasting and Dynamic Models; Durbin and Koopman, Time Series Analysis by State Space Methods; Harvey on structural time series.
- Hierarchical and empirical-Bayes shrinkage: Efron and Morris on shrinkage; Gelman and Hill on multilevel models.
- Conformal and permutation-style validity (relevant to the order-statistic threshold): Vovk, Gammerman and Shafer, Algorithmic Learning in a Random World.
- Canary and deployment analysis systems: Netflix/Google Kayenta documentation; industrial reports on automated canary analysis (verify authors and venues).
- Statistics for the evaluation: Efron (1979, bootstrap); Holm (1979, sequential Bonferroni); Benjamini and Hochberg (1995); Kaplan and Meier (1958); work on restricted mean survival time; Gelman et al. on pre-registration and multiple comparisons; Clopper and Pearson (1934).
- Sparse multi-stream detection beyond Xie-Siegmund (for example Chan and Walsh on sparse mixtures); verify before citing.

Novelty check. The research report and its audit concluded that none of dynamic factors, hierarchy, sparse selection, persistence, and online change detection is new individually, and that Barigozzi et al. is close prior art for combining factors and change points. Before the introduction says 'to our knowledge', Antigravity runs a structured literature search (queries listed in docs/litsearch.md, results with dates), and the author reads the closest ten papers. The paper claims a *conditional empirical comparison and protocol*, not a new model family.

5.8 Claims discipline: allowed and disallowed statements

Allowed (if the evidence exists)	Not allowed
In a declared simulation, at validated equal false-alert rates, method X detected class-wide +3% harm faster than Y by Z minutes [CI]	Method X is better than classical methods (general)

Allowed (if the evidence exists)	Not allowed
The hierarchy outperformed independent hosts in class-wide harm and not in sparse harm	The hierarchy is the best model
Thresholds calibrated on plain nulls did not transfer to [environment]; matched recalibration restored validity	The detector is robust to misspecification (unless every perturbation passes the gate)
Under a concurrent workload step the false-alert rate was [range]; abstention limited it to [range] at a cost of [minutes]	The detector handles workload shifts
On [real trace] placebo rollouts the false-alert rates were [range]; injected effects were detected with power [range]	The method works on real production telemetry
Calibration is exact in finite samples under exchangeability (beta-binomial)	The method controls false discoveries (host-level model-based FDR is not frequentist control)
Attribution assumes uncontaminated controls and no interference; we measured the cost of violating it	The canary/control contrast establishes causal attribution

Interpretation rules. (1) A method with a looser achieved rate looks faster: never rank unless every method passes the gate. (2) A contrast inside the equivalence margin is a tie. (3) Report where the hierarchy loses. (4) 'Persistence helps' needs the fair ablation. (5) Higher persistence can retain unrelated slow drift and delay recovery: show the trade-off. (6) Posterior probabilities are exact only conditional on parameters and model; say so. (7) Simulator agreement with the model is a threat, not a result.

5.9 Threats to validity

Type	Threat	Mitigation in this plan
Construct	Capped time and the 15-minute event rule may not match operators' costs	Decision-theoretic analysis (E7); event definition sensitivity (D in {10, 15, 30}, κ in {log 1.01, log 1.02, log 1.05})
Internal	Tuning asymmetry across methods	Equal-effort protocol, grid expansion, budget-parity study E3, oracles as bounds
Internal	Leakage of truth or test information	AST lint, role separation, seed registry, prereg tag before test generation
Internal	Implementation bugs	Independent references (statsmodels), regression tests per bug, second implementation of key statistics (for example scalar KF versus matrix KF), mutation checks
Conclusion	Multiple comparisons, small effects	Holm, equivalence margin, sample-size planning (F2)
External	Simulator matches model; single metric; Gaussian; injected effects only	G-extensions, R-studies, real-trace placebo and injection, fidelity table, calibrated simulation
External	Environment shift (calibration transfer)	Transfer and recalibration studies R2, small-B recalibration (WP-C6)
Statistical	Order-statistic threshold variability	Exact beta-binomial validity test; B and n sized by the test's power

5.10 Reproducibility package

- Repository tag for the paper (paper-v1) with the lock file; the pre-registration tag (prereg-v1) and the frozen model tag.
- One command per figure and table: `python -m canary_ssm.cli reproduce --target F6` regenerates it from stored CSV; `reproduce --from-scratch --scale small` regenerates a reduced-size study end to end in under an hour.
- Data statement (no real data distributed; download scripts; licences), compute statement (machine, threads, wall time, energy estimate optional), random-seed statement (registry), software statement (versions).
- Artifact: the repository, the manifests, the CSV outputs, the figure scripts. Include a README that states what is and is not reproducible (for example large real traces must be downloaded).

5.11 Writing workflow

1. Write the paper in LaTeX (or the target venue's template). All numbers enter the text through macros generated from CSV: report.py emits results_macros.tex with entries such as \newcommand{\HierVsIndepDiff}{−0.2~min [−2.9, 2.7]}. A hand-typed number in the .tex is a bug; a lint script greps for digits outside macros in results sections.
2. Tables are emitted by report.py as booktabs LaTeX; figures as vector PDF.
3. Citations: only from the verified list (Section 5.7) with DOI recorded in a .bib file generated from docs/references.csv; the coding agent and I must never invent a citation. Every citation is checked against its DOI landing page.
4. Use the coding agent for code, tests and table generation; use it for prose only for drafting from the generated tables, and the author edits every claim against the claims table (5.8).
5. Internal review checklist (Appendix E) before sharing; ask a reader unfamiliar with the project to try to break the calibration argument.

5.12 Venue options (hedged)

I cannot verify current venue scopes or deadlines from here; check each before committing. The material fits (a) applied data science or systems-for-ML tracks that value benchmarks and protocols; (b) software-engineering and operations venues concerned with deployment safety and AIOps; (c) statistics or quality-engineering journals if the calibration theory and sequential-detection comparison are emphasised; (d) workshops on AIOps or reliability for an early version. Match the paper's framing to the branch of Section 1.6: a method paper suits (a); a benchmark/protocol paper suits (a) or (b); the calibration theory suits (c).

Part 6. Execution run book

6.1 Order of work

Estimates below assume Antigravity writes the code, you review, and the PC runs jobs unattended overnight. They are planning estimates with wide error bars, not measurements. Effort is calendar time part-time.

Phase	Steps (in order)	Estimated effort	Gate to proceed
P0 Housekeeping	WP-I1 (git, lock, task runner); WP-A3 manifest; WP-A4 seeds; WP-I2 parallel runner and cache; WP-I6 on-the-fly; WP-I3 first tests; X-0.1 reproduction and determinism	3 to 6 days	Reproduction identical; workers=1 equals workers=8; manifest complete
P1 Protocol repair	WP-A1 regimes; WP-A2 per-family validation; WP-A5, A6 studies package; WP-B1-B6 grids and selection; WP-C1-C4 calibration; X-0.2 anomalies; X-0.3 grid expansion; X-0.4 plain-regime validity	1.5 to 2.5 weeks	Every primary method passes G2 in every plain family; anomalies explained
P1b Baselines	WP-E1 baselines (read sources, implement, test); WP-E2 oracles; WP-B4 fair ablation methods	1 to 2 weeks (overlaps P1)	All baselines pass X-0.4 or are dropped with reason
P2 Fair benchmark	Tag prereg-v1 (WP-F1); WP-F2-F9 statistics and reports; run E1, E2, E3, E4, E5, E6, E7, E8	2 to 3 weeks	Decision-tree branch chosen and written (docs/DECISION.md)
P3 Methods	WP-D1 EB slopes; WP-D2 controls-based noise; WP-D3 RS-SSM; WP-D4 support policy; WP-D5 robust; WP-D7 scores; WP-D8 enumeration; WP-D9 averaging; each unit-tested before any benchmark	2 to 4 weeks (can overlap P2 after P1)	Each option has a pass/fail on its adoption criterion in a study
P3b Robustness	Simulator extensions G2, G3, G4, G5, G6, G8 as needed; run R1 to R6; R7 scalability; R8 class structure; R9 optional	3 to 5 weeks	Failure map complete for the three failures of the first run (step, φe and tails, loading/shock)
P4 Real traces	Inspect sources; loader; RT-1 placebo; RT-2 injection; RT-3 fidelity; calibrated-simulation rerun of E1	2 to 4 weeks (depends on data access)	At least one real trace set analysed, or an explicit statement that none was usable
P5 Paper	Theory notes T1 to T5 verified; macros and figures; draft; claims table; reproducibility package; internal review	3 to 5 weeks	Every number traces to a CSV; reproduce target works from scratch at small scale

Total: roughly 10 to 18 weeks part-time if phases overlap sensibly and the real-trace phase finds usable data. If it does not, the paper can still be written without P4 as a simulation-and-protocol paper, but the claims table must say so.

6.2 Compute plan for the PC (12 threads, 16 GB RAM)

- Workers: 10. Set OMP_NUM_THREADS, MKL_NUM_THREADS, OPENBLAS_NUM_THREADS to 1 in each worker. Keep the machine free of other heavy jobs during runs; disable sleep; plug in power.
- Memory: each worker holds one rollout (about 1 MB) plus detector state; 16 GB is ample. Cache directories live on the fastest local disk; clear caches when the nuisance hash changes.
- Disk: follow WP-I6. Keep a 20 GB margin. The main saved roles (dev 600, calibration 2000, validation 14,000, test 15,000 rollouts) can be stored (about 22 GB) or regenerated; prefer regenerating validation and test.
- Runtime planning table (estimates):

Job	Rollouts	Single-core estimate	10 workers (estimate)
X-0.3 fit with grids_v2 on 500 dev pairs	1000 × (about 60 candidates × 3 SSM families + baselines)	10 to 20 h	1.5 to 3 h, less with the residual cache
X-0.4 calibration (2000) + validation (14,000)	16,000	3.5 h	30 to 40 min

Job	Rollouts	Single-core estimate	10 workers (estimate)
E1 main benchmark	about 45,000	10 h (15 h with D3, D8, D9)	1.3 to 2 h
E2 ablations	about 8 ablations × 2400	11 h	1.5 h
E4 power surfaces	36,000 pairs = 72,000	16 h	2 h
R1 (b) grid	about 94,000	21 h	2.7 h
R2	about 218,000	48 h	6 h
R3 to R6	about 150,000 together	33 h	4 h

Run long jobs with --resume and a progress journal. Interrupted runs must restart without redoing completed chunks.

6.3 Risk register

Risk	Likelihood / impact	Mitigation
Windows multiprocessing failures (spawn, pickling, BLAS oversubscription)	High / medium	Top-level functions only; thread env vars; test workers=1 vs 8 early (WP-I2)
Agent edits a test or weakens a definition to get green	Medium / high	Rule 2 and rule 12 of Section 0.1; review diffs of tests/ on every merge; protected branch
Truth leakage into a new detector module	Medium / high	Extended AST lint (WP-I3); oracles isolated
Fair benchmark erases the SSM advantage	Medium / medium (scientific, not failure)	Decision tree (1.6); the protocol paper stands either way
Calibration cannot be made valid for a method	Medium / medium	Drop with stated reason; investigate exchangeability; never keep it silently
Real-trace data unusable or inaccessible	Medium / medium	State the limitation; rely on calibrated-simulation fidelity checks; place real-trace in future work
Simulator extensions change legacy behaviour	Medium / medium	Bit-identity tests for defaults; new RNG streams appended
Compute time larger than planned	High / low	Reduce cell counts using adaptive stopping; prioritise R1 and R2 first; cache residuals
Interpretation drift in write-up	Medium / high	Macros from CSV; claims table; independent reader
Scope creep (R9 multi-metric, D3 on 100 canaries)	High / medium	Mark as stretch; freeze scope at the end of P2

6.4 If-then decisions

Observation	Action
A method fails X-0.4 in one family	Check exchangeability (KS), nuisance fit on the wrong data, eligibility differences; fix or drop; do not widen the gate
Best classical baseline within 3 minutes of best SSM in E1	Report a tie; follow the negative-result branch of Section 1.6; keep the protocol and failure map as the contribution
Hierarchy ties independent hosts everywhere	Examine X-0.2(c) and R8; if the partition is uninformative say so; consider D8 as the structure-adaptive contribution
D8 or D9 recovers the best structure per family	Make it the headline method; rerun E1 contrasts including it with prereg amendment tagged before test regeneration
No variant passes R1 at the centre cell	State the limit; make abstention (P1) the recommendation; report abstention cost curve
Matched recalibration restores validity in R2	The recommendation is per-environment recalibration; quantify the history needed (WP-C6)
Real-trace placebo rate far above 5% for all methods	The simulator's null is too clean: refit simulator to real statistics, rerun, and report both
Results differ between Windows and Linux beyond 1e-9	Stop; find the nondeterminism (BLAS, summation order); fix or record tolerance

Part 7. Prompt library for Antigravity

Send prompts one at a time. Every prompt assumes the standing instruction below was given at the start of the session and that this PDF (saved as docs/Canary_SSM_Change_Plan.pdf) and the repository's AGENTS.md are in the workspace. Each prompt names a section of this document; the agent must read that section first.

P-00 Standing instruction (paste first in every new session)

Antigravity prompt
You are working in the canary_ssm_project repository. Read AGENTS.md and Part 0 of docs/Canary_SSM_Change_Plan.pdf now and follow them for the whole session.
Key rules: one work package or experiment at a time; never edit or weaken an existing test; never invent numbers (all numbers come from CSV produced by commands you ran); PASS/FAIL computed by code; detector code never reads ground truth; causal scores only; roles never mix; frozen models are for audits only; seeds from canary_ssm/seeds.py; report failures; stop after the task and wait.
Before coding, restate in five lines what you will change and which tests you will add. After coding, run python -m pytest -q and paste the summary. If anything in the document is ambiguous or seems wrong, stop and ask.

Housekeeping and protocol

ID	Prompt
P-A3/A4	Implement WP-A3 (manifest, git init, tag v0.1-first-full-run, requirements.lock) and WP-A4 (seeds.py, registry tests). Replace every ad-hoc seed. Show the manifest of a 20-pair smoke run and the output of test_seed_ranges_disjoint.
P-A5/A6	Implement WP-A5 and WP-A6: create canary_ssm/studies, port experiment_7 and experiment_8 faithfully (same logic) with registry seeds and require_tuned guards, add the study CLI. Verify with legacy seeds at n=50 that counts match the first run's tables within Monte Carlo noise. Do not apply any fix yet.
P-B1/B2	Implement WP-B1 (grids_v2.yaml, candidate_grid reading it, grid hash) and WP-B2 (edge_selection.csv, warnings, --strict-edges). Do not run a full fit; run on a 30-pair toy development set and show edge_selection.csv.
P-B3	Implement WP-B3 (K-fold selection, one-standard-error rule, objective weighting). Add the three tests. Show on a toy dev set that held-out nulls are excluded from each fold's threshold.
P-B4/B5/B6	Implement WP-B4 (fair no-persistence grid, random-walk variant), WP-B5 (cutoff range 1..n and host AUC), WP-B6 (tuned/frozen guard). Add tests. Do not run benchmarks.
P-C1/C2	Implement WP-C1 (policy schema v2 with regimes, reading schema 1 still works) and WP-C2 (stored maxima, thresholds_for_alpha, threshold_uncertainty). Add the hand-value and Beta-quantile simulation tests.
P-C3	Implement WP-C3: betabinom_validity in metrics.py, gate G2 in report.py, exchangeability audit (KS and Mann-Whitney). Show the validity p-value for (45 of 500, B=500, k=476) and (42 of 500) and for synthetic exchangeable data. Test uniformity of p-values under exchangeability.
P-C5/C6	Implement WP-C5 (recalibrate command, --refit-nuisance) and WP-C6 (min_B_for_alpha, conservative order statistic). Add tests. No studies yet.
P-I2	Implement WP-I2 (parallel.py, residual cache, resume, thread env vars) and WP-I6 (on-the-fly generation). Prove workers=1 and workers=8 produce identical CSV and that on-the-fly equals from-disk. Report speed-ups measured on a 200-rollout job.

Methods and baselines

ID	Prompt
P-D2	Implement WP-D2 (fit_noise_from_controls, noise_source options). Report phi estimation error versus truth for true phi in {0.3, 0.6, 0.8, 0.9}. Do not change defaults.
P-D5	Implement WP-D5 (update_robust and hier_t) with the three tests. Show that with nu=1e6 the output matches the Gaussian filter to 1e-6.
P-D7	Implement WP-D7 score alternatives (union bound, Monte Carlo any-host with MC standard error, posterior mean max, class-aware). Register as options; report rank correlations on 200 harmed rollouts.
P-E1	Implement the baselines of WP-E1 one at a time, each in its own commit. For Xie-Siegmund, BOCPD and mSPRT first read the cited source, write the statistic with equation numbers in docs/baselines.md, then implement exactly that and add a hand-computed fixture test. Stop and ask if a statistic cannot be reproduced from the source.

ID	Prompt
P-E2	Implement oracles.py (WP-E2) as the only module allowed to import Truth. Label every output ORACLE and exclude from rankings. Add the leakage-lint exemption and a test that detector modules still cannot import Truth.

Statistics, simulator and studies

ID	Prompt
P-F1/F2	Create docs/PREREG.md and configs/prereg.yaml from the proposed contrasts (do not tag yet). Implement stats.py: paired bootstrap by pair id, Holm adjustment, sign-flip permutation test, equivalence verdicts. Add tests against hand-computed examples.
P-F3/F4	Implement Kaplan-Meier cumulative incidence with Greenwood intervals, RMST, competing-risk handling of premature alarms, and power-versus-alpha curves with AUC. Test KM and RMST on a hand-computed toy dataset.
P-F5/F6/F7	Implement host AUC, Bayesian FDR selection, reliability curves with Wilson intervals and Brier decomposition, the expected-loss analysis with cost-ratio sweep, and the advisory-policy grid. Test each on small synthetic cases with known answers.
P-F9	Harden report.py: assert_report_numbers, figure functions with CSV twins, claims table, LaTeX macro and booktabs table emitters. Test on a fixture results directory.
P-G	Implement simulator extensions of WP-G one at a time in order G2, G3, G4, G5, G6, G8, G1, G7 (R1 needs G2 and G6 first). For each: new fields with legacy defaults, bit-identity test, paired-invariant test, truth-isolation test.
P-X0.3	Run X-0.3 (grid expansion) on 500 plain development pairs with the parallel runner. Report edge flags, expansions performed, the frozen model's manifest, and the development profile plots. Do not touch calibration or test data.
P-X0.4	Run X-0.4 exactly as specified (2000 calibration, 2000 per family validation). Produce all listed outputs. If any method fails G2, investigate non-exchangeability per Section X-0.2(b) and report; do not change the gate.
P-E2-E8	Run E2 ablations, E3 sensitivity, E4 power surfaces, E5 power-alpha, E6 localization, E7 decision loss, E8 flicker as separate sessions, each from the frozen model and policy. Each session ends with tables only.
P-R2	Run R2 for phi_e, heavy tails, loading shift and shock first (arms A to F as available). Produce transfer curves and the minimal-repair table from CSV. No tuning.
P-R3-R9	Run R3, R4, R5, R6, R7, R8 as separate sessions; R9 only if the multi-metric extension was approved.
P-RT	Follow Part 4. First write docs/data_sources.md after inspecting each candidate source and record schemas and licences. Stop and report before writing any loader if no suitable data is found.
P-AUDIT	Independent audit of the repository against Part 0 rules: truth leakage, test edits (diff tests/ against tag v0.1-first-full-run), calibration/validation role mixing, tie and strict-exceedance handling, eligibility, seeds, PASS/FAIL strings not traceable to numbers, report numbers not matching CSVs. Return defects with file and line evidence. Do not fix in the same step.

Appendices

Appendix A. Mathematical reference

A.1 Kalman filter (Joseph form, current implementation)

```
predict:  m- = F m ;  P- = F P F' + Q
update :  v = r_obs - H m- ;  S = H P- H' + V ;  K = P- H' S^-1  (Cholesky solve)
          m = m- + K v ;  P = (I - K H) P- (I - K H)' + K V K' ;  P <- (P + P')/2
log-likelihood increment:  -0.5 ( log|S| + v' S^-1 v + p log 2 pi )
Missing data: update only the observed coordinates; if none, predict only. Gap of k steps: k predictions.
```

A.2 Threshold and exact validity

```
B null maxima M_1..M_B (exchangeable, continuous). k = ceil((B+1)(1-alpha)). Threshold T = M_(k).
F(T) ~ Beta(k, B+1-k)  =>  exceedance probability p = 1 - F(T) ~ Beta(B+1-k, k)
Alerts K among n independent validation nulls:  K ~ BetaBinomial(n, a = B+1-k, b = k)
E[K] = n a / (a+b) ;  Var[K] = n a b (a+b+n) / ((a+b)^2 (a+b+1))
Example: B=500, alpha=0.05, k=476, n=500: a=25, b=476, E[K]=24.95, SD[K]=6.88.
P(K >= 45) = 0.0060 ;  P(K >= 42) = 0.0148.
A valid method fails the rule 'CP upper bound <= 0.08' with probability 0.286 at B=n=500; 0.0875 at B=n=1000; 0.0064 at B=n=2000.
Policy's true false-alert rate  ~ Beta(25, 476) for B=500: 95% range [0.0326, 0.0706].
```

These probabilities were computed with scipy.stats.betabinom in the sandbox; Antigravity must recompute them in a test (test_betabinom_mean_sd_hand_values) before relying on them.

A.3 Binomial intervals

```
Clopper-Pearson (exact), level 1-a:  lower = Beta^-1(a/2; K, n-K+1) ;  upper = Beta^-1(1-a/2; K+1, n-K)
One-sided upper for K=0:  1 - a^(1/n)  (n=60, a=0.05: 0.0487)
Wilson interval (for reliability bins): centre (p + z^2/2n)/(1+z^2/n), half-width z sqrt(p(1-p)/n + z^2/4n^2)/(1+z^2/n)
```

A.4 Empirical-Bayes slope shrinkage (WP-D1)

```
per host:  y = mu + beta u + e ;  s^2 = c sigma^2 / sum u^2 ;  c = (1+phi)/(1-phi)
class g controls:  beta_bar = sum(b_j / s_j^2) / sum(1/s_j^2) ;  tau^2 = max(var(b_j) - mean(s_j^2), tau_floor^2)
canary i:  w = tau^2/(tau^2 + s_i^2) ;  beta_tilde = w b_i + (1-w) beta_bar ;  var_post = w s_i^2
```

A.5 Regression-state SSM (WP-D3)

```
state s = [theta (d), e (n), delta (3n)] ; delta_i = (mu_i - mu_tilde_i, beta_i - beta_tilde_i, lam_i - lam_tilde_i)
F = blockdiag(diag(rho), diag(phi), I) ; Q = blockdiag(diag(sd^2), diag(q^2), 0)
H_t(i,:) = [L_i, e_i', 0.., (1, u_t, fhat_t) at block i, 0..] ; V_t = lam lam' U_t
P_0 = blockdiag(prior_sd^2 I, P_e, blockdiag(kappa_Sigma * diag(se_mu^2, var_post_beta, var_post_lam)))
```

A.6 Support statistics (WP-D4)

```
S_out(t) = max(0, x_t - x_max, x_min - x_t) / s_pre ;  S_shift(t) = |mean(x_{t-4..t}) - x_bar_pre| / s_pre
s0 = 99.9th percentile over development plain nulls of max_t max(S_out, S_shift)
```

A.7 Hypothesis enumeration (WP-D8)

```
hypotheses h in {0, singletons, classes, all}; filters F_h with effect map L_h.
log w_{h,t} = log( (1-eps) w_{h,t-1} + eps/K ) + ell_{h,t} - ell_{0,t}   (then normalise)
host risk p_i = sum_{h: i in S_h} pi_h Pr_h(theta_h > kappa | data) ; release score = log max_i p_i
```

A.8 Sequential statistics for baselines (verify against sources)

```
CUSUM:    C_t = max(0, C_{t-1} + z_t - k)
EWMA:     S_t = alpha z_t + (1-alpha) S_{t-1}
Prewhitened residual: e_t = z_t - phi z_{t-1}   (a persistent shift delta becomes (1-phi) delta)
Window GLR (mean shift, unit variance): max_{s} (sum_{j=s..t} z_j)^2 / (2 (t-s+1))
mSPRT (Gaussian mixture N(0,tau^2)): Lambda_t = sqrt(s^2/(s^2 + t tau^2)) exp( tau^2 S_t^2 / (2 s^2 (s^2 + t tau^2)) )
   always-valid p-value: p_t = min_{u<=t} min(1, 1/Lambda_u)
Xie-Siegmund mixture GLR, BOCPD: take equations from the papers; do not reconstruct from this table.
```

A.9 Inference for comparisons

```
Paired difference d_p = capped_A(p) - capped_B(p), pairs p = 1..n.
Bootstrap: resample pair ids with replacement (B=10,000), percentile CI of mean(d).
Sign-flip permutation p-value: flip signs of d_p at random 20,000 times; p = P(|mean(d*)| >= |mean(d)|).
Holm: sort p_(1)<=...<=p_(m); adjusted p_(i) = max_{j<=i} min(1, (m-j+1) p_(j)).
Equivalence (TOST logic): equivalent if the 95% CI of mean(d) lies within +/- margin (3 minutes).
Sample size: n = ((z_{1-a/2} + z_{power}) SD / delta)^2 ; z=1.96, 0.84 -> n = (2.8 SD/delta)^2.
```

A.10 Survival and loss

```
RMST(tau) = integral_0^tau S(t) dt ; here capped time = RMST of time-to-success censored at the remaining horizon.
Kaplan-Meier S(t) = prod_{t_i<=t} (1 - d_i/n_i) ; Greenwood var = S^2 sum d_i / (n_i (n_i - d_i)).
Brier decomposition: BS = reliability - resolution + uncertainty.
Expected loss(r) = r * P(false alert) + E[harm-minutes]   (r = C_fa / C_miss)
```

Appendix B. Configuration examples

```
# configs/plain_main.yaml (regime plain; no step)
regime: plain
sim: {sd_host: 0.02}
roles:
  dev: {pairs: 500}
  calibration: {pairs: 2000}
  null_validation: {per_family: 2000}
  test: {pairs: 1000}
families:
  - {name: S1_sparse,       weight: 0.24, overrides: {n_affected: 3,  effect_pct: 3.0, duration: 45}}
  - {name: S2_class_wide,   weight: 0.24, overrides: {affected_mode: class0, effect_pct: 3.0, duration: 45}}
  - {name: S3_broad,        weight: 0.12, overrides: {n_affected: 10, effect_pct: 3.0, duration: 45}}
  - {name: S4_weak_sparse,  weight: 0.12, overrides: {n_affected: 3,  effect_pct: 2.5, duration: 45}}
  - {name: S5_label_errors, weight: 0.12, overrides: {affected_mode: class0, effect_pct: 3.0, duration: 45, label_error: 0.3}}
  - {name: single_canary,   weight: 0.12, overrides: {n_affected: 1,  effect_pct: 4.0, duration: 45}}
  - {name: transient,       weight: 0.04, overrides: {n_affected: 3,  effect_pct: 3.0, duration: 10}}
```

```
# configs/grids_v2.yaml (version string recorded in manifest)
version: v2.0
ssm: {rho: [0.9, 0.95, 0.99, 0.995, 0.999], sd_g: [0.0005, 0.001, 0.002, 0.005], prior_sd: [0.01, 0.02, 0.03],
      constrain_long_run_sd_le_kappa: true}
ssm_no_persist: {rho: [0.0], sd_g: [0.005, 0.01, 0.02, 0.03, 0.05], prior_sd: [0.01, 0.03, 0.05], constrain: false}
adj_cusum: {k: [0.1, 0.25, 0.5, 0.75, 1.0, 1.5]}
adj_ewma: {alpha: [0.02, 0.05, 0.1, 0.2, 0.35, 0.5]}
topk_cusum: {K: [1, 2, 3, 5, 7, 10], k: [0.25, 0.5, 0.75, 1.0]}
matched_contrast: {window: [1, 5, 10, 15, 30]}
localization_cutoff: {range: [1, 10]}
```

```
# configs/prereg.yaml (skeleton; tag prereg-v1 BEFORE generating test data)
alpha: 0.05
margin_minutes: 3.0
horizon: 120
primary_contrasts:
  C1: {a: hier_ssm, b: indep_ssm}
  C2: {a: hier_ssm, b: pooled_ssm}
  C3: {a: best_ssm_by_dev, b: best_baseline_by_dev}
  C4: {a: hier_ssm, b: hier_fair_no_persist}
  C5: {a: hier_ssm, b: hier_rank0}
  C6: {a: hier_ssm, b: hier_white}
  C7: {a: hier_ssm@S5, b: hier_ssm@S2}
  C8: {a: structure_adaptive, b: best_of_hier_indep}
scenarios: [S1_sparse, S2_class_wide, S3_broad, S4_weak_sparse, S5_label_errors, single_canary]
noise_levels: [0.01, 0.02, 0.04]
decision_rule: docs/DECISION.md (branch chosen from E1 results)
multiplicity: holm
bootstrap: {resamples: 10000, unit: pair_id}
exploratory: everything not listed above
```

Appendix C. Command-line map

Command	Status	Purpose
simulate --config --role --out [--csv]	exists; add per_family, regime, registry seeds	Generate role data (or manifest only with on-the-fly)
fit --data --out --methods --beta-mode [--frozen] [--strict-edges] [--grid]	exists; extend	Nuisance fit and K-fold selection on development data
calibrate --data --model --out --regime [--B]	exists; extend	Regime thresholds, stored maxima, multi-alpha
validate-null --data --model --policy	exists; extend	Per-family validity with beta-binomial test
audit-null --config --model --policy --n --out	exists	Misspecification audits on fresh nulls

Command	Status	Purpose
recalibrate --model --data --regime [--refit-nuisance] --out	new (WP-C5)	New thresholds from new healthy history
evaluate --data --model --policy --out [--generate-on-the-fly]	exists; extend	Test-set evaluation (refuses frozen models and regime mismatch)
study NAME --model --policy --out	new (WP-A6)	Run R1 to R9, E1 to E8, X studies
reproduce --target F6 | --from-scratch --scale small	new	Regenerate a figure/table or a reduced study
benchmark	exists	Runtime scaling

Appendix D. Result file dictionary (columns that must exist)

File	Key columns
per_rollout_metrics.csv	rollout_id, pair_id, family, regime, harm, method, kind (no_event|transient|event), status, alarm_time, onset, delay, capped, followup, brier, elig_frac
table_validation_by_family.csv	family, method, alerts, n, rate, ci_lo, ci_hi, expected_alerts, bb_sd, bb_pvalue_two_sided, above_target
gate_status.json	criterion, regime, cells, evaluable_cells, cells_ok, failing_cells_expected_by_chance, status (PASS|FAIL|NOT_EVALUABLE), failing[]
exchangeability_audit.csv	method, ks_stat, ks_p, mw_p, n_cal, n_val
calibration_alpha_sweep.csv	method, alpha, k, B, threshold, achieved_rate_validation, ci
threshold_uncertainty.csv	method, alpha, threshold, boot_lo, boot_hi, far_beta_lo, far_beta_hi
edge_selection.csv	method, parameter, selected, grid_min, grid_max, at_edge, grid_version
table_paired_differences.csv	contrast_id, A, B, scenario, noise, mean_diff, ci_lo, ci_hi, holm_p, equivalent, n_pairs
study_R*/cells.csv	study, factor columns, method, arm, policy, n_null, alerts, rate, ci, bb_p, n_harm, success30, capped, abstain_frac
experiment_manifest.json	commit, dirty, config_sha, regime, beta_mode, noise, grid_version, grid_sha, candidate_table_sha, selected, edge_flags, seeds, versions, machine, timings, failures, hash chain

Appendix E. Checklists

E.1 Before any run

- Tests pass; working tree clean or dirty flag recorded; config and grid hashes recorded; seeds from registry; regime correct; frozen/tuned status correct; disk estimate below 70% of free space; resume enabled; thread env vars set.

E.2 Before reading any result

- gate_status.json read first; validation tables with beta-binomial p read; fit failures and abstention read; manifest inspected; no PASS without evaluable cells.

E.3 Before writing a claim

- Every compared method passed the gate in that regime; contrast is pre-registered or labelled exploratory; interval and equivalence margin reported; stratified results checked for reversals; limitations paragraph updated; claim is in the allowed column of Section 5.8.

E.4 Before submission

- Every number traces to a CSV via macros; reproduce target works from scratch at small scale; every citation checked against its DOI; related-work novelty search dated and archived; threats to validity table complete; data and software statements written; an independent reader tried to break the calibration argument; negative and null results included with equal prominence.

Appendix F. Glossary

Term	Meaning
Canary / control	Hosts running the new build / the old build during a rollout
Adjusted residual r	Log latency minus per-host intercept, workload term and factor term; the stream every detector consumes
Eligibility	A minute is eligible if enough canaries are observed and enough fresh controls exist; ineligible minutes cannot alarm (predict-only)
Release score	A scalar per minute whose maximum over the horizon is compared with a calibrated threshold
Latched alarm	The first alarm ends the rollout's detection outcome; later scores are ignored
Capped time	Delay for successes, remaining horizon for everything else; the restricted mean of time to success
Regime	plain (no concurrent workload shift) or step (a shared workload shift concurrent with the rollout); calibrated separately
Exchangeability	Calibration and validation maxima are identically distributed and independent; the premise of the exact validity test
Pivotal statistic	A statistic whose null distribution does not depend on nuisance parameters, so thresholds transfer
Oracle	A detector given ground truth; an upper bound, never a competitor
Frozen model	A model with declared, untuned hyperparameters, used only for calibration audits

Appendix G. Pre-registration template (docs/PREREG.md)

```
Title / version / date / git tag
1. Research questions and directional hypotheses (H1..Hn) with the scenario and noise levels each refers to
2. Data roles and sizes; seed purposes; generation configs (hashes)
3. Methods in scope; tuning protocol; grids (hash); edge-expansion rule; selection objective
4. Calibration protocol: regime, B, alpha, validity gate (beta-binomial p >= 0.01 and rate <= 0.075), what happens on failure
5. Primary outcomes: capped time (per scenario, per noise), miss/premature/late rates, success at 15/30/60
6. Primary contrasts (configs/prereg.yaml), margin of practical equivalence, multiplicity method
7. Exclusion rules (none for failures; all reported)
8. Exploratory analyses (explicitly listed)
9. Decision rule mapping outcomes to the paper branch (Section 1.6)
10. Deviations log (filled during the study; each deviation timestamped and justified)
```

Appendix H. Repository map after the changes

```
canary_ssm/
  schema.py simulate.py seeds.py manifest.py parallel.py
  factor.py preprocess.py support.py adaptive (noise from controls) kalman.py
  hierarchy.py rs_ssm.py enum_ssm.py bma.py
  baselines.py baselines_extra.py oracles.py (ORACLE only)
  decision.py metrics.py stats.py semisynth.py report.py experiments.py cli.py
  studies/ study_regime.py study_robust.py study_E*.py study_R*.py
configs/ plain_main.yaml step_regime.yaml grids_v2.yaml prereg.yaml audit.yaml studies/*.yaml
docs/ ARCHITECTURE.md DESIGN_DECISIONS.md DECISION.md PREREG.md baselines.md data_sources.md litsearch.md wp/*.md
tests/ (existing 41 tests unchanged) + new tests per work package
scripts/ tasks.ps1 tasks.sh smoke.sh audit_quick.sh run_all.ps1
```

Appendix I. Troubleshooting

Symptom	Likely cause and action
Run is slower with more workers	BLAS thread oversubscription: set thread env vars to 1 in worker initialiser
Workers crash on Windows with pickling errors	Lambdas or nested functions passed to the pool; use top-level functions and pass paths
Gate NOT_EVALUABLE	Per-family n below minimum; generate per-family validation (WP-A2)
All thresholds infinite	Fewer than 19 nulls at alpha = 0.05 (k exceeds B): increase B
Policy hash mismatch	Model files changed after calibration: recalibrate
Regime mismatch error	Policy and data regimes differ; use the right data or pass --allow-regime-mismatch for audits only
Calibration and validation maxima differ (KS rejects)	Not exchangeable: check generator settings, seeds, nuisance fitted on one set, family mix mismatch

Symptom	Likely cause and action
Method identical to another to many digits	Check whether an option actually changes the model (for example white noise pinned at the floor); compare nuisance files
Null table empty or NaN	String labels read by pandas as missing (the kind='null' bug); use explicit labels and keep_default_na=False in CSV readers